

Robogeбра

Project-Based Interview Pack

Senior / 5+ years Full-Stack · Angular · Java 17 · Spring Boot · MongoDB · Microservices · AWS

Every answer tied to real project code

Robogeбра — Project-Based Interview Pack

Target: Senior / 5+ years Full-Stack (Angular · Java 17 · Spring Boot · MongoDB · Microservices · AWS)

Idea: Every answer is tied to **real code in the Robogeбра project** — so when an interviewer asks "*where did you use X in your real project?*", you have a concrete file, pattern, and story.

How to use this pack

- Each file = one interview area. Read the **Question** → **Answer** → "**In Robogeбра**" structure.
- The "**In Robogeбра**" block is your talking point — memorize the *flow*, not the code character-for-character.
- Say answers **out loud**. If asked to go deeper, you can reference the file path.

The sections

Core areas (your requested format):

#	File	Covers
1	01-frontend.md	Angular — memory leaks/ takeUntil , RxJS, change detection, zone.js vs zoneless, DI, guards, resolvers, interceptors, forms, state, performance
2	02-backend.md	REST API design, controllers, layering, request→auth lifecycle, the "email already exists" flow
3	03-java.md	OOP (Entity↔Model↔DTO), Collections & ArrayList, Streams, Java 17 features
4	04-spring.md	Spring Boot, IoC/DI, Spring Data, @Transactional , validation, config, @Async / scheduling
5	05-database.md	MongoDB — documents, indexes, @DBRef , repositories, the precedence-query pattern
6	06-microservices.md	Architecture, portal↔CRM communication, service boundaries, whiteboard diagram
7	07-aws.md	Cognito (3 pools), S3, SES/SNS/Lambda, signup + JWT auth

Deep dives (the details interviewers push on):

#	File	Covers
8	08-error-handling.md	Global @ControllerAdvice , all custom exceptions, error-code enums, 3rd-party error remap, filter-level errors
9	09-auth-deep-dive.md	Authentication (10-filter chain, dual JWT, 3 Cognito pools) + Authorization (@PermissionContext , role/permission model, validateAsTeacher)
10	10-annotations-reference.md	Every annotation used (Spring/Lombok/Mongo/validation/Swagger) + your custom @interface s

Extra topics to crack the interview:

#	File	Covers
11	11-testing.md	JUnit 5 + Mockito + Testcontainers (backend), Jasmine/Karma (frontend), test strategy
12	12-system-design.md	Whiteboard walkthroughs: solution-visibility, S3 upload, notifications, scaling
13	13-performance-caching.md	Caffeine cache, Mongo indexing, pagination, @DBRef N+1, frontend perf
14	14-behavioral-pitch.md	60-sec intro, project pitch, STAR stories from real features , questions to ask, Git/CI-CD

Key project facts (know these cold)

- **robogebra-web** — student/teacher web app — **Angular 16.2**, NgModule, **zone.js** (not zoneless), no NgRx (BehaviorSubject-based state)
- **robogebra-backoffice** — admin/ops UI — Angular 16.2
- **robogebra-mobile** — **Ionic 8** (Angular + Capacitor), Angular 18.2
- **robogebra-portal** — main backend — **Spring Boot 3.2**, **Java 17**, **Maven**, **MongoDB** ([robogebra_cms_db](#))
- **robogebra-admin-portal** — second Spring Boot backend for AI/content generation
- **robogebra-crm** — **Node.js/Express** microservice (institutes, coupons, subscriptions), MongoDB ([robogebra-coupon-mgmt](#))

- **robogebra-crm-web** — **React 18 + Vite**
- **robogebra-website** — static (Parcel)
- **Auth** — AWS **Cognito** (3 user pools: user / email-user / institute-user) + **JWT** + OAuth2 (Google/LinkedIn)
- **Architecture** — microservices hybrid; portal (3000) ↔ CRM (3001) over REST

Honesty note (important for senior interviews)

The project is **Angular 16 with zone.js** — it is **not** zoneless and does **not** use signals or standalone components yet. When asked about zoneless/signals, answer as: *"We're on Angular 16 with zone.js and OnPush-where-needed; here's how zoneless/signals work and how I'd migrate."* Interviewers respect the honest "here's our reality + here's the modern direction" answer far more than pretending.

1. Front-End (Angular) — Interview Q&A tied to Robogebra

Stack: Angular 16.2 (`robogebra-web` , `robogebra-backoffice`), Angular 18 + Ionic 8 (`robogebra-mobile`).
NgModule architecture, zone.js, RxJS-heavy, service-based state (no NgRx).

Q1. How do you prevent memory leaks in Angular? *(the #1 question)*

Answer. Memory leaks in Angular almost always come from **RxJS subscriptions that outlive their component**. If you `subscribe()` and never unsubscribe, the observable keeps a reference to the component, so it's never garbage-collected, and its callback keeps firing after the view is gone. I handle it three ways, in order of preference:

1. **async pipe** in the template — Angular subscribes and unsubscribes automatically.
2. **takeUntil(destroy\$)** for imperative subscriptions — a `Subject` I complete in `ngOnDestroy`.
3. Manual `Subscription.unsubscribe()` only for one-off cases.

In Robogebra. The `takeUntil` + `destroy$ / unsubscribe$` pattern is our standard, in basically every component. Example — `robogebra-web/src/app/app.component.ts`:

```
export class AppComponent implements OnInit, OnDestroy {
  private destroy$: Subject<void> = new Subject<void>();

  ngOnInit(): void {
    this.authService.isLoggedInObservable()
      .pipe(takeUntil(this.destroy$))
      .subscribe(isLoggedIn => { /* handle */ });
  }

  ngOnDestroy(): void {
    this.destroy$.next(); // emits → all takeUntil pipes complete
    this.destroy$.complete(); // releases the Subject itself
  }
}
```

A subtle real one — even our **HTTP interceptor** implements `OnDestroy` and cancels in-flight requests with a `cancelRequests$` subject (`header-authorization.interceptor.ts`), so a token-refresh retry can't leak. And in the template we lean on the `async` pipe (`app.component.html`) so those subscriptions clean themselves up:

```
<ng-container *ngIf="isLoggedIn() | async"> ... </ng-container>
```

Follow-up: "How would you enforce it?" In Angular 16+ you can use `takeUntilDestroyed()` (from `@angular/core/rxjs-interop`) to drop the boilerplate `destroy$` Subject — that's the direction I'd move us toward.

Q2. Explain the RxJS operators you actually use, and why.

Answer. The ones I reach for most and *why*:

- `switchMap` — cancel the previous inner observable when a new value arrives. Perfect for "latest wins" like search or token refresh.
- `combineLatest` — combine the latest values of several streams into a derived view.
- `debounceTime` — rate-limit rapid user input before an expensive call.
- `BehaviorSubject` — hold state with a current value; expose read-only via `asObservable()`.
- `catchError` — recover/rethrow inside a stream.

In Robogebra.

`switchMap` for silent token refresh (`header-authorization.interceptor.ts`) — if the access token expired, refresh it and *switch* the original request onto the new token:

```
return this.authService.refreshToken().pipe(
  filter(newAuth => !!newAuth),
  switchMap(newAuth => {
    const refreshed = req.clone({ setHeaders: { Authorization: `Bearer ${newAuth.token}` } });
    return next.handle(refreshed).pipe(takeUntil(this.cancelRequests$));
  }),
  catchError(err => { this.authService.setSessionExpired(true); return throwError(() => err); })
);
```

`combineLatest` to decide UI visibility from two streams (`app.component.ts`):

```
return combineLatest([
  this.exerciseSolutionService.getFullScreenStateAsObservable(),
  this.anonymousLessonService.getAnonymousCodeAsObservable()
]).pipe(map([isFullScreen, anonCode] => !isFullScreen && !anonCode),
  takeUntil(this.destroy$));
```

`debounceTime(500)` + `switchMap` to throttle expensive worksheet execution (`exercise-solution-executor.component.ts`):

```
this.executeWorksheetSubject$.pipe(
  debounceTime(500),
  switchMap(payload => this.solutionExecutorService.execute(payload)),
  takeUntil(this.unsubscribe$)
).subscribe();
```

Follow-up: `switchMap` vs `mergeMap` vs `concatMap`? `switchMap` cancels the previous (latest wins — search, refresh); `mergeMap` runs all in parallel (independent writes); `concatMap` queues them in order (sequential, order matters). I use `switchMap` for the token refresh precisely because I *want* to abandon a stale refresh.

Q3. How does change detection work, and do you use OnPush?

Answer. Angular's default change detection runs a dirty-check over the whole component tree whenever something *might* have changed — triggered by zone.js patching async APIs (events, timers, XHR).

`ChangeDetectionStrategy.OnPush` narrows that: the component is only checked when an `@Input`

reference changes, an event fires in it, or an observable it uses (via `async`) emits — which is a big perf win on large trees.

In Robogebra (honest). Our apps run the **default** strategy, not `OnPush`. Where we need to force a refresh imperatively — e.g. after a payment callback that happens outside Angular's awareness — we inject `ChangeDetectorRef` and call `detectChanges()` (`payments.component.ts`):

```
constructor(private cdr: ChangeDetectorRef) {}  
onPaymentComplete(): void { this.updatePaymentStatus(); this.cdr.detectChanges(); }
```

What I'd improve. Heavy list components (chapter/exercise lists, analytics) are good `OnPush` candidates — combined with the `async` pipe and immutable updates, that would cut redundant checks significantly. That's a concrete perf story I can tell.

Q4. Zone.js vs zoneless — what's the difference, and what does your project use?

Answer. zone.js monkey-patches async browser APIs so Angular *knows* when to run change detection — it's the "magic" behind not having to tell Angular when data changed. **Zoneless** (stable in Angular 19, experimental in 18) removes that: you drop the `zone.js` polyfill and use `provideZonelessChangeDetection()`; Angular then relies on **signals**, the `async` pipe, and explicit `markForCheck()` to know when to update. Benefits: smaller bundle (no `zone.js`), faster, and no "ran outside Angular" surprises.

In Robogebra (honest). We're **zone-based**. `robogebra-web` bootstraps the classic way and ships `zone.js ~0.13`:

```
// main.ts  
platformBrowserDynamic().bootstrapModule(AppModule).catch(err => console.error(err));
```

No `provideZonelessChangeDetection` anywhere. So my honest answer is: "Our app is Angular 16 with `zone.js`. I understand zoneless and where it's going — to migrate I'd first move hot components to `OnPush` + `signals`, replace manual subscriptions with the `async` pipe, then flip the zoneless provider and remove the polyfill." That shows I know it without pretending we shipped it.

Q5. Standalone components & Signals — do you use them?

Answer (honest). Not yet — our codebase is **100% NgModule** (`AppModule`, `SharedModule`, `CoreModule`, `FeaturesModule`) and predates signals. I know both: - **Standalone components** remove `NgModules` — a component declares its own `imports`. Simpler, better tree-shaking, `bootstrapApplication()` instead of `bootstrapModule()`. - **Signals** — a reactive primitive (`signal()`, `computed()`, `effect()`) that makes change detection precise and is the foundation of zoneless.

In Robogebra. `app.module.ts` shows the classic structure — feature modules + `APP_INITIALIZER`:

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, HttpClientModule, AppRoutingModule, SharedModule, FeaturesModule,
CoreModule, ...],
  providers: [{ provide: APP_INITIALIZER, useFactory: initializeAnonymousLesson, deps:
[AnonymousLessonService], multi: true }],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

That `APP_INITIALIZER` is itself a good talking point — we use it to bootstrap anonymous-lesson state *before* the app renders.

Q6. How does Dependency Injection work in your app?

Answer. Angular DI is hierarchical. I make services singletons with `@Injectable({ providedIn: 'root' })` — tree-shakeable, one instance app-wide — and inject them via the constructor. For things scoped to a module (like interceptors) I register them in that module's `providers`.

In Robogebra. `auth.service.ts`:

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  constructor(private router: Router, private http: HttpClient,
    private notificationService: NotificationService,
    private busySpinnerService: BusySpinnerService, ...) {}
}
```

The interceptor uses plain `@Injectable()` (no `providedIn`) because it's registered in the module provider array with the `HTTP_INTERCEPTORS` multi-token.

Q7. How are routes protected? Explain your auth guard.

Answer. With **route guards** — `CanActivate` decides if a route may be entered. My guard returns an `Observable<boolean>`: it checks login state reactively, and on failure it **stashes the attempted URL** and redirects to login so the user lands back where they wanted after signing in. It also enforces **role-based** access.

In Robogebra. Wired in `app-routing.module.ts`:

```
{ path: AppRoutes.DASHBOARD, component: DashboardComponent,
  canActivate: [AuthGuardService],
  resolve: { [AppResolver.USER_SUMMARY]: UserSummaryResolver,
    [AppResolver.SUBSCRIPTION_PLAN]: SubscriptionPlanResolver } }
```

The guard (`core/guard/auth-guard.service.ts`):

```

@Injectables({ providedIn: 'root' })
export class AuthGuardService implements CanActivate {
  canActivate(route, state): Observable<boolean> {
    return this.authService.isLoggedInObservable().pipe(
      tap(isLoggedIn => {
        if (!isLoggedIn) {
          this.authService.setRedirectUrl(state.url); // remember where they wanted to go
          this.router.navigate(['/login']);           // redirect on failure
        }
      }),
      map(isLoggedIn => {
        if (!isLoggedIn) return false;
        if (this.authService.isLoggedInAsParent()) { /* restrict parent to allowed routes */ }
        return true;
      })
    );
  }
}

```

The full chain: route has `canActivate: [AuthGuardService]` → guard asks `AuthService` for login state → if false, save URL + redirect → if true (and role allowed), the route activates. It's async (returns an Observable) so it works even before the auth state has resolved.

Q8. Where do you use Route Resolvers, and why?

Answer. A **resolver** pre-fetches data *before* a route activates, so the component never renders in a half-loading state — the data is ready in `route.data` on `ngOnInit`. I also use it to drive a global loading spinner around the navigation.

In Robogeбра. `core/resolvers/chapter.resolver.ts` pre-loads a chapter before its page opens:

```

@Injectables({ providedIn: 'root' })
export class ChapterResolver {
  resolve(route, state): Observable<ChapterModel> {
    this.resolverService.startResolver(); // show spinner
    const chapterId = route.params['chapterId']; // from URL /chapter/:chapterId
    return this.chapterListService.fetchChapterDetailsById(chapterId, false)
      .pipe(finally(() => this.resolverService.endResolver())); // hide spinner
  }
}

```

Wired with `resolve: { [AppResolver.CHAPTER]: ChapterResolver }`, and the component reads it:

```

this.route.data.subscribe(data => this.chapterData = data[AppResolver.CHAPTER]);

```

Flow: navigate to `/chapter/123` → router runs `ChapterResolver` → waits for the HTTP call → *then* activates `ChapterComponent` with data already present. We use resolvers for chapter details, user summary, subscription plan, and domain config on the dashboard/login routes.

Trade-off I'd mention: resolvers delay navigation until data arrives, which can feel slow on a bad network — so I only resolve *critical* above-the-fold data and lazy-load the rest inside the component.

Q9. Explain your HTTP interceptor.

Answer. An `HttpInterceptor` sits in the pipeline for every request/response. Mine does three jobs: (1) attach the `Authorization: Bearer` header and default `Content-Type`, (2) centralize error handling with `catchError`, and (3) transparently **refresh an expired token and retry** the original request — so components never deal with 401s.

In `Robogebra` (`header-authorization.interceptor.ts`):

```
intercept(req, next): Observable<HttpEvent<any>> {
  let headers = req.headers;
  if (!headers.has('Content-Type') && !(req.body instanceof FormData))
    headers = headers.set('Content-Type', 'application/json');
  const auth = this.authService.authDetails;
  if (auth) headers = headers.set('Authorization', `Bearer ${auth.token}`);
  return this.invokeHttpCall(req.clone({ headers }), next);
}
private invokeHttpCall(req, next) {
  return next.handle(req).pipe(
    takeUntil(this.cancelRequests$),
    catchError(err => this.handleUnAuthResponse(err, req, next)) // 401 → refresh + retry
  );
}
```

Note the **immutability** — `HttpRequest` is read-only, so I `clone()` it to add headers. That's a classic interview point.

Q10. Reactive forms & custom validators?

Answer. I use **reactive forms** (`FormBuilder` / `FormGroup`) for anything non-trivial — they're typed, testable, and support dynamic + cross-field validation. For cross-field rules (like password match) I put a **group-level validator**; for conditional rules I swap validators at runtime with `setValidators()` + `updateValueAndValidity()`.

In `Robogebra`. Signup form (`signup.component.ts`) with a group-level password-match validator:

```
this.signupFormGroup = this.formBuilder.group({
  displayName: ['', [Validators.required]],
  userName:    ['', [Validators.required]],
  password:    ['', [Validators.required]],
  confirmPassword: ['', [Validators.required]],
  otp: ['', [Validators.required]]
}, { validators: this.passwordMatchValidator() });
```

Dynamic validators — the username field validates as **phone or email** depending on the chosen login mode (`auth-validation.service.ts`):

```
if (loginBy === LoginBy.PHONE_NUMBER)
  userNameCtrl.setValidators([Validators.required,
    this.phoneNumberValidator(getCountryCode())]);
else
  userNameCtrl.setValidators([Validators.required, this.emailValidator()]);
userNameCtrl.updateValueAndValidity();
```

We also react to `valueChanges` (with `takeUntil`) to toggle UI, e.g. detect a phone vs email input live.

Q11. Parent-child component communication?

Answer. `@Input()` for parent→child data, `@Output() EventEmitter` for child→parent events. For sibling/cross-tree communication I use a shared service with a `BehaviorSubject`.

In Robogebra. The renewal modal (`renew-confirmation-modal.component.ts`) takes config via inputs and emits the decision:

```
@Input() finalAmount: number;
@Input() couponCode: string;
@Output() onPaymentConfirmation = new EventEmitter<boolean>();
onContinue(): void { this.onPaymentConfirmation.emit(true); }
```

```
<app-renew-confirmation-modal [finalAmount]="amount"
(onPaymentConfirmation)="handlePayment($event)">
</app-renew-confirmation-modal>
```

Q12. How do you manage state without NgRx?

Answer. With **services holding `BehaviorSubject` s** — the "service with a subject" pattern. Private subject (has a current value), exposed read-only via `asObservable()`, updated only through setter methods. It gives me single-source-of-truth and reactive consumers without NgRx's boilerplate. NgRx makes sense when state is large, shared, and needs time-travel/devtools; for our scale, services are simpler.

In Robogebra (`auth.service.ts`):

```
private loggedInBSubject$ = new BehaviorSubject<boolean>(false);
private sessionExpiredBSubject$ = new BehaviorSubject<boolean>(false);

isLoggedInObservable(): Observable<boolean> { return this.loggedInBSubject$.asObservable(); }
setSessionExpired(v: boolean): void { this.sessionExpiredBSubject$.next(v); }
```

Consumers can *read* but can't `next()` into the subject — encapsulation. Same pattern in the mobile app (`exercise-solution-slide.service.ts`).

Q13. Performance optimizations you've applied?

Answer. (1) `trackBy` on every meaningful `*ngFor` so Angular reuses DOM nodes instead of destroying/recreating on list changes; (2) the `async` pipe to avoid manual subscription overhead and leaks; (3) **lazy-loaded** route modules; and going forward, **OnPush** on heavy trees.

In Robogebra. `trackBy` in the analytics/progress lists (`chapter-progress-list.component.ts`, `student-analytics.component.ts`):

```
trackByChapterId(_i: number, chapter: ChapterProgressModel): string { return chapter.id; }
```

```
<div *ngFor="let chapter of chapters; trackBy: trackByChapterId">{{ chapter.name }}</div>
```

And `async` -pipe-driven template sections in `app.component.html` so those streams never need manual cleanup.

Q14. How do you show backend errors to users? (e.g. "email already exists")

Answer. I never let raw HTTP errors reach the component. The service `catchError` s, hands the `HttpErrorResponse` to a central `NotificationService` , which extracts the backend's `error.message` and shows a toast. This keeps error UX consistent everywhere.

In Robogeбра. Signup path — component triggers a subject, service does the POST, error flows to notifications:

```
// auth.service.ts
createUser(payload): Observable<string> {
  return this.http.post(url, payload, { responseType: 'text' }).pipe(
    catchError(err => this.throwErrorMessage(err)), // → NotificationService
    map(r => r)
  );
}
private throwErrorMessage(err: HttpErrorResponse): Observable<any> {
  this.busySpinnerService.setSaveInProgress(false);
  this.notificationService.handleErrorResponse(err, null, true, true); // shows toast
  return EMPTY; // swallow → stream completes cleanly
}
```

`NotificationService` prioritizes `error.message` from the backend (e.g. "An account with this email already exists." — returned by the portal) and toasts it for ~5s. **End-to-end:** duplicate email → backend `APIException` → JSON `{message}` → interceptor/service `catchError` → `NotificationService` toast → spinner stops. (Backend side of this exact flow is in `07-aws.md` / `02-backend.md`.)

Rapid-fire recap

Topic	Robogeбра pattern	File
Memory leaks	<code>takeUntil(destroy\$)</code> + <code>async</code> pipe	<code>app.component.ts</code> , interceptor
RxJS	<code>switchMap</code> token refresh, <code>combineLatest</code> , <code>debounceTime</code>	interceptor, executor
Change detection	default + manual <code>detectChanges()</code>	<code>payments.component.ts</code>
Zoneless	No — zone.js, Angular 16 (know the migration)	<code>main.ts</code>
DI	<code>providedIn: 'root'</code> + constructor injection	<code>auth.service.ts</code>
Guards	<code>CanActivate</code> → save URL + redirect	<code>auth-guard.service.ts</code>
Resolvers	pre-fetch chapter before route	<code>chapter.resolver.ts</code>
Interceptor	header + 401 refresh/retry	<code>header-authorization.interceptor.ts</code>
Forms	reactive + group/dynamic validators	<code>signup.component.ts</code>
State	service + <code>BehaviorSubject</code> (no <code>NgRx</code>)	<code>auth.service.ts</code>
Perf	<code>trackBy</code> , <code>async</code> , lazy load	progress/analytics lists

2. Back-End (API design & request lifecycle) — Q&A tied to Robogebra

This section is the *backend thinking* layer — how a request flows end to end, how the layers are organized, status codes, and the two flows you asked about: the **auth/JWT chain** and "**email already exists**". (Java-language and Spring-annotation depth are in [03-java.md](#) / [04-spring.md](#).)

Stack: Spring Boot 3.2 monolithic [robogebra-portal](#) (port 3000), domain-driven packages, MongoDB.

Q1. Walk me through the lifecycle of an authenticated request in your backend.

Answer. Filters → controller → service → repository → DB, then DTO back out:

1. Request hits the **security filter chain**. `JwtAuthenticationFilter` (a `OncePerRequestFilter`) extracts the `Bearer` token, validates it (SSO or Cognito), loads the user, sets the Spring `SecurityContext`, and **stashes the user on the request as an attribute**.
2. The **controller** method runs; it reads that user via `@RequestAttribute` and validates the `@RequestBody` with `@Valid`.
3. The **service** does business logic + authorization (`teacher.validateAsTeacher()`), calling other services/repositories.
4. The **repository** (Spring Data Mongo) reads/writes documents.
5. The result is mapped to a **DTO** and serialized to JSON. Any exception is caught by the `@ControllerAdvice` and turned into a consistent error body.

In Robogebra — the controller reading the filter-provided user:

```
public List<SolutionVisibilityDTO> updateStatus(  
    @RequestAttribute(ApplicationConstants.INSTITUTE_USER_ATTRIBUTE) InstituteUser  
    instituteUser,  
    @Valid @RequestBody SolutionVisibilityUpdateDTO request) { ... }
```

The `instituteUser` was never sent by the client — the **filter** put it there after authenticating. That decoupling (filter authenticates, controller consumes) is the key design idea.

Q2. Explain the auth/JWT validation chain in detail. (*you asked how auth connects*)

Answer. A single `OncePerRequestFilter` centralizes authentication so no controller repeats it. Steps: extract token → pick validation path (our own SSO JWT vs AWS Cognito) → load the user → build a Spring `Authentication` and set it in `SecurityContextHolder` → put the user (and institute) on request attributes → continue the chain. On token errors it writes a clean error response instead of letting a 500 escape.

In Robogebra (`security/JwtAuthenticationFilter.java`):

```

@Component @Order(6) @AllArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Override protected boolean shouldNotFilter(HttpServletRequest req) {
        return hasAnnotationWithBypassJwtAuth(req); // skip @BypassJwtAuth endpoints
        (webhooks, CRM)
    }

    @Override protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
        FilterChain chain) {
        try {
            String token = authService.getTokenFromRequest(req);
            if (needTokenValidation(req, token)) {
                JwtAuth jwtAuth = authService.isSSOToken(token)
                    ? authService.getAuthDetailsBySSOToken(token) // our JWT (issuer check)
                    : authService.getAuthDetailsByToken(token); // AWS Cognito validation

                UserDetails user = userDetailsService.loadUserByUsername(getUserName(jwtAuth));
                var authToken = new UsernamePasswordAuthenticationToken(user, null,
                    user.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(authToken);

                req.setAttribute(ApplicationConstants.LOGGED_USER_ATTRIBUTE, jwtAuth);
                addInstituteAttributes(req, jwtAuth.getId()); // sets INSTITUTE_USER_ATTRIBUTE
            }
            chain.doFilter(req, res);
        } catch (CognitoException e) { responseWriter.writeCognitoException(res, req, e); }
    }
}

```

Cognito token validation itself (`AuthService.getAuthDetailsByToken`) maps Cognito SDK exceptions to HTTP statuses — `NotAuthorizedException` → 401 , `TooManyRequestsException` → 400 — so auth failures are *typed*, not generic 500s.

Why this design? One place owns authentication; controllers stay clean; the `@BypassJwtAuth` annotation lets service-to-service calls (CRM, webhooks) skip user auth and use a service token instead. That's the standard senior answer: *cross-cutting concern* → *filter, not scattered checks*.

Q3. Walk through "user signs up with an email/phone that already exists." (you asked for this)

Answer. The signup endpoint delegates to a service that tries to **find-or-create** the user. If the account already exists (and is already a full Cognito account), it throws a business exception with **HTTP 400** and a human message; the global advice turns that into a clean JSON error the frontend shows as a toast. It also handles the legitimate case of *linking* Cognito to an existing SSO (Google/LinkedIn) user.

In Robogebra.

Controller (`auth/controller/AuthController.java`) — thin, delegates + documents the 400:

```
@PostMapping("/create-user")
@ApiResponses({ @ApiResponse(responseCode = "400", description = "Bad Request") }) //
duplicate
public ResponseEntity<String> createUser(@Valid @RequestBody CreateUserRequestDTO dto) {
    authService.createUser(dto.toModel());
    return new ResponseEntity<>("Successfully send OTP to the " + dto.getUserName(),
HttpStatus.OK);
}
```

Service (`auth/service/AuthService.java`) — find-or-create + duplicate detection:

```
private User resolveOrCreateUser(String userName, ExternalUser externalUser, UserRole role,
String referral) {
    return userPersistenceService.findByUserName(userName)
        .map(entity -> linkCognitoToExistingUser(entity, externalUser.getExternalId(),
userName))
        .orElseGet(() -> createNewUserWithDefaults(externalUser, userName, role, referral));
}

private User linkCognitoToExistingUser(UserEntity user, String cognitoId, String userName) {
    if (isSsoUser(user) && user.getCognitoId() == null) { // valid: link Cognito to
Google/LinkedIn user
        userPersistenceService.linkCognitoAccount(user.getId(), cognitoId);
        return user.toModel();
    }
    // DUPLICATE: real account already exists
    throw new APIException(HttpStatus.BAD_REQUEST, "An account with this email already
exists.");
}
```

Global advice returns the structured error → frontend `NotificationService` shows the toast (frontend side in `01-frontend.md` Q14).

Nice detail to mention: the check is `findByUserName(...).map(link).orElseGet(create)` — the whole find-or-create is expressed with `Optional`, no null branches. And it distinguishes "*link to existing SSO user*" (allowed) from "*true duplicate*" (rejected) — a real product nuance, not just a blind uniqueness check.

Q4. How do you design your REST resources and status codes?

Answer. Noun-based, hierarchical URLs (`/api/solution-visibility/exercise-items/{id}/solution-access`), correct verbs (GET read, POST create, PATCH partial update), DTOs in/out, and meaningful statuses: 200 OK, 400 validation/business error, 401 unauthenticated, 403 forbidden, 404 missing, 500 unexpected. Errors share one shape `{status, message, timestamp, path}`.

In Robogebra. URLs encode the hierarchy Book→Chapter→Exercise→ExerciseItem; the visibility check is nested under its item: `GET /api/solution-visibility/exercise-items/{exerciseItemId}/solution-access`. `PATCH /status` is a partial update (not PUT) because we only change status for selected classes.

Q5. How do you keep the service layer clean and testable?

Answer. Thin controllers, business logic in services, DB access behind a persistence service/repository, immutable constructor-injected deps. Authorization lives in the service (`validateAsTeacher()`), not the

controller, so it's enforced regardless of entry point. That separation means I can unit-test a service by `new`-ing it with mocked collaborators.

In Robogebra — the visibility service validates role + target existence before doing work, then streams the update per class:

```
public List<SolutionVisibility> updateStatus(InstituteUser teacher, SolutionVisibilityLevel
level,
                                           String targetId, List<String> klassIds,
SolutionVisibilityStatus status) {
    teacher.validateAsTeacher();           // authZ in the service
    validateTargetExists(level, targetId); // business validation
    return klassIds.stream().distinct()
        .map(k -> updateStatusForKlass(teacher, k, targetId, status))
        .collect(Collectors.toList());
}
```

Rapid-fire recap

Topic	Robogebra example	File
Request lifecycle	filter → controller → service → repo → DTO	<code>JwtAuthenticationFilter.java</code>
Auth chain	JWT/Cognito → SecurityContext → <code>@RequestAttribute</code>	<code>JwtAuthenticationFilter.java</code> , <code>AuthService.java</code>
Duplicate signup	find-or-create + <code>APIException(400)</code>	<code>AuthService.java</code>
REST/status codes	nested nouns, PATCH, uniform error body	<code>SolutionVisibilityController.java</code>
Clean layering	authZ in service, thin controller	<code>SolutionVisibilityService.java</code>

3. Java (Core) — Interview Q&A tied to Robogebra

Stack: Java 17, Maven, in `robogebra-portal` (main backend) and `robogebra-admin-portal`.

Q1. Explain the four OOP principles with real examples from your project.

Answer (one-liners first, then a real Robogebra example for each):

- **Encapsulation** — bundle data + behavior, hide internals; expose intent.
- **Abstraction** — program to a contract, hide the "how".
- **Inheritance** — share a skeleton via a base class.
- **Polymorphism** — same call, different behavior at runtime.

1) Encapsulation — `SolutionVisibility` model

Fields are private; callers get **behavior**, not raw state. The status comparison is hidden behind `isEnabled()`, and conversions are methods on the object — nobody outside reaches in:

```
// domain/solutionvisibility/model/SolutionVisibility.java
public class SolutionVisibility {
    private String klassId;
    private String targetId;
    private SolutionVisibilityStatus status;           // internal representation, hidden

    public boolean isEnabled() { return this.status == SolutionVisibilityStatus.ENABLED; }
    public SolutionVisibilityEntity toEntity() { return
SolutionVisibilityEntity.builder()...build(); }
    public SolutionVisibilityDTO toDTO() { return
SolutionVisibilityDTO.builder()...build(); }
}
```

At the **architecture level**, the **Entity** ↔ **Model** ↔ **DTO** trio is encapsulation too: Entity = MongoDB shape, Model = business logic, DTO = API shape. Each hides its concern, so the DB schema, internal logic, and public contract can each change independently. Same idea in `InstituteUser.validateAsTeacher()` — the role rule is encapsulated on the object, not duplicated by every caller.

2) Abstraction — repository & service interfaces

I program to contracts and let Spring supply the implementation. `SolutionVisibilityRepository` is just an **interface** — I declare *what* I want (`findByClassIdAndTargetId`), Spring Data generates the *how*:

```
// repository/SolutionVisibilityRepository.java
@Repository
public interface SolutionVisibilityRepository extends
MongoRepository<SolutionVisibilityEntity, String> {
    Optional<SolutionVisibilityEntity> findByClassIdAndTargetId(String klassId, String
targetId);
    List<SolutionVisibilityEntity> findByClassIdInAndTargetIdIn(List<String> klassIds,
List<String> targetIds);
}
```

Same with the `NotificationService` interface — callers depend on the abstraction, never on Email/SMS/WhatsApp concretely (see polymorphism below).

3) Inheritance — service-auth filter hierarchy

All three service-to-service auth filters share one base that extends Spring's `OncePerRequestFilter`, so they inherit the whole validation flow and only fill in specifics:

```
// security/AbstractServiceAuthenticationFilter.java
public abstract class AbstractServiceAuthenticationFilter extends OncePerRequestFilter {
    protected abstract Class<? extends Annotation> getAnnotationClass();
    protected abstract String getHeaderName();
    protected abstract String getExpectedToken();
    protected abstract UserRole getUserRole();
    protected void additionalValidation(HttpServletRequest req) { /* default no-op hook */ }
    // doFilterInternal(): applies? → validateTokenExists → validateToken →
    additionalValidation → authenticate
}

// CrmServiceAuthenticationFilter, InternalServiceAuthenticationFilter,
// WebhookAuthorizationFilter
// each `extends AbstractServiceAuthenticationFilter` and inherits the algorithm
```

That's a **template method** pattern: the base owns the algorithm, subclasses supply the steps.

4) Polymorphism — notification channel resolved at runtime

`NotificationService` is an interface with `EmailNotificationService`, `SmsNotificationService`, `WhatsAppNotificationService` implementations. A **factory** returns the right one by looking at the username, and the caller invokes `sendOTP(...)` **without knowing which concrete type** it holds — the correct override runs at runtime:

```
// domain/notification/service/NotificationServiceFactory.java
@Async
public void sendOtp(String userName, String otp) {
    getNotificationService(userName).sendOTP(userName, otp); // polymorphic call
}
private NotificationService getNotificationService(String username) {
    return username.matches(RegexPatternUtils.USER_NAME_PHONE_NUMBER_PATTERN)
        ? whatsappNotificationService // phone → WhatsApp impl
        : emailNotificationService; // else → Email impl
}
```

Add a new channel later → implement the interface, no caller changes. Method-level polymorphism also shows in every `toModel()` / `toEntity()` override and the abstract filter's `getUserRole()` per subclass.

Two-line summary to say: "Encapsulation is our Entity/Model/DTO objects hiding state behind methods like `isEnabled()`; abstraction is our repository/service interfaces; inheritance is the `AbstractServiceAuthenticationFilter` base the CRM/internal/webhook filters extend; polymorphism is the `NotificationService` interface where a factory picks WhatsApp vs Email and the caller just calls `sendOTP()`."

Q2. Collections — where do you use List/Map/Set and ArrayList?

Answer. `List` / `ArrayList` for ordered collections (query results, ids); `Map` / `HashMap` for lookups and grouping; `Set` for uniqueness. I favor the `List` interface as the declared type (program to interface) and immutable `List.of(...)` for fixed small lists.

In **Robogebra**. A real `Map` built by grouping, then used for fast lookup — the solution-visibility access check (`SolutionVisibilityService.java`):

```
// Group DB rows into a Map<targetId, rows> for O(1) precedence lookup
Map<String, List<SolutionVisibilityEntity>> rowsByTarget =
    solutionVisibilityPersistenceService.findByClassIdsAndTargetIds(classIds, precedence)
        .stream()
        .collect(Collectors.groupingBy(SolutionVisibilityEntity::getTargetId));

// Immutable ordered List for the precedence walk (most specific first)
List<String> precedence = List.of(exerciseItemId, exerciseId, chapterId);
for (String targetId : precedence) {
    List<SolutionVisibilityEntity> rows = rowsByTarget.get(targetId);
    if (rows != null && !rows.isEmpty())
        return rows.stream().anyMatch(r -> r.getStatus() == SolutionVisibilityStatus.ENABLED);
}
```

This is a great "why this data structure" story: I fetch all candidate rows in **one** DB call, `groupingBy` into a `Map` so each precedence lookup is $O(1)$, and walk an **ordered immutable List** so the most-specific rule wins. Alternative (a DB call per level, or nested loops over a `List`) would be N queries / $O(n^2)$.

Follow-up: `ArrayList` vs `LinkedList`? `ArrayList` = array-backed, $O(1)$ random access, cache-friendly — my default. `LinkedList` only wins for frequent head/middle inserts. In practice I've never needed `LinkedList` here; stream `.collect(Collectors.toList())` gives an `ArrayList`.

Follow-up: `HashMap` internals? Array of buckets; key `hashCode()` → bucket, `equals()` resolves collisions; since Java 8 a bucket becomes a balanced tree past 8 entries ($O(\log n)$ worst case). That's why entity keys need correct `hashCode` / `equals`.

Q3. Java Streams — show real pipelines and explain them.

Answer. Streams give me declarative, composable data processing. Key pieces: `map` (transform), `filter` (keep), `flatMap` (flatten nested), `distinct`, terminal `collect` / `toList` / `anyMatch`.

In **Robogebra**.

map + distinct + collect — apply a visibility update to each class, de-duped (`SolutionVisibilityService`):

```
return classIds.stream()
    .distinct()
    .map(klassId -> updateStatusForKlass(teacher, klassId, targetId, status))
    .collect(Collectors.toList());
```

anyMatch — short-circuit "is it enabled for *any* of the student's classes":

```
return rows.stream().anyMatch(r -> r.getStatus() == SolutionVisibilityStatus.ENABLED);
```

flatMap (nested streams) — build caches from a grouped config (`CacheConfig`):

```
List<CaffeineCache> caches = cacheProperties.getManager().stream()
    .flatMap(group -> group.getCacheNames().stream()
        .map(name -> new CaffeineCache(name, Caffeine.newBuilder()
            .expireAfterWrite(group.getTtlMinutes(), TimeUnit.MINUTES)
            .maximumSize(group.getMaxSize()).build())))
    .toList();
```

map on a Page — convert entities to models while preserving pagination ([SolutionStepFeedbackService](#)):

```
return repository.findByReviewedUser_Id(userId,
    pageable).map(SolutionStepFeedbackEntity::toModel);
```

Follow-up: intermediate vs terminal / laziness? [map](#) / [filter](#) / [flatMap](#) are lazy intermediates — nothing runs until a terminal ([collect](#) , [toList](#) , [anyMatch](#) , [forEach](#)) pulls values. [anyMatch](#) also **short-circuits**, so it stops at the first match.

Q4. Which Java 17 features do you use, and where?

Answer. We're on **Java 17 LTS**. The ones I actually use:

Switch expressions (arrow syntax) — no fall-through, exhaustive, can return a value. Dispatch by enum without break-bugs ([SolutionVisibilityService](#)):

```
switch (level) {
    case CHAPTER      -> chapterService.validateExistence(targetId);
    case EXERCISE      -> exerciseService.validateExistence(targetId);
    case EXERCISE_ITEM -> {
        if (Boolean.FALSE.equals(exerciseItemService.validateExerciseItem(targetId)))
            throw new APIException(HttpStatus.NOT_FOUND, "Exercise item not found.");
    }
}
```

As a **value** ([ExceptionHandlerAdvice](#)), mapping an error type to an HTTP status — note grouped cases:

```
HttpStatus status = switch (ex.getErrorType()) {
    case INVALID_COUPON -> HttpStatus.BAD_REQUEST;
    case SERVICE_UNAVAILABLE, LAMBDA_ERROR, NETWORK_ERROR -> HttpStatus.SERVICE_UNAVAILABLE;
    case PARSING_ERROR -> HttpStatus.INTERNAL_SERVER_ERROR;
};
```

Optional — no null-checks; express "maybe absent" in the type ([RefreshTokenService](#)):

```
return refreshTokenRepository.findById(userId)
    .map(UserRefreshTokenEntity::getRefreshToken)
    .orElse(null);
// elsewhere: .orElseThrow(() -> new ResourceNotFoundException("Solution Step Feedback", id));
// and: repository.findById(userId).ifPresent(entity -> { ... });
```

.toList() (Java 16+) — a concise, immutable terminal that replaces [Collectors.toList\(\)](#) ([SolutionVisibilityController](#)):

```
return service.updateStatus(...).stream().map(SolutionVisibility::toDTO).toList();
```

`var` for local inference, **text blocks** for multi-line strings/JSON, and I understand **records** (immutable data carriers) and **sealed classes** (restricted hierarchies) even where our older DTOs still use Lombok classes.

Follow-up: records vs Lombok? A `record` is a concise, *immutable* data class with auto `equals/hashCode/toString`. Our DTOs use Lombok `@Builder` / `@Getter`/`@Setter` because they're **mutable** (builders, framework deserialization). If I were writing new immutable DTOs I'd reach for `record`.

Q5. `equals()` / `hashCode()`, immutability, `final` — quick senior checks.

Answer. - `equals` / `hashCode` **contract** — equal objects must have equal hash codes; override both together. Critical for `HashMap` / `HashSet` keys (see Q2 grouping). - **Immutability** — `final` fields + no setters → thread-safe, cacheable. Our injected dependencies are all `private final` (constructor injection), which is immutability applied to services. - `final` — on fields (assign once), params (no reassignment), classes/methods (no override).

In Robogebra. Every service holds `private final` collaborators:

```
private final SolutionVisibilityPersistenceService solutionVisibilityPersistenceService;  
private final StudentEnrollmentService studentEnrollmentService;
```

`final` + Lombok `@AllArgsConstructor` = constructor injection with immutable, non-reassignable dependencies — safe to share across threads.

Rapid-fire recap

Topic	Robogebra example	File
OOP (Entity↔Model↔DTO)	<code>toEntity()/toModel()/toDTO()</code>	<code>SolutionVisibility.java</code>
Abstraction/ Polymorphism	abstract filter + template method	<code>AbstractServiceAuthenticationFilter.java</code>
Collections/Map	<code>groupBy</code> + <code>List.of</code> precedence	<code>SolutionVisibilityService.java</code>
Streams	<code>map/distinct/collect</code> , <code>anyMatch</code> , <code>flatMap</code>	service, <code>CacheConfig.java</code>
Java 17 switch	arrow + value-returning switch	service, <code>ExceptionHandlerAdvice.java</code>
Optional	<code>map/orElse/orElseThrow/</code> <code>ifPresent</code>	<code>RefreshTokenService.java</code>
Immutability	<code>private final</code> deps	every <code>@Service</code>

4. Spring / Spring Boot — Interview Q&A tied to Robogebra

Stack: Spring Boot 3.2, Java 17, Maven, Spring Data MongoDB, Spring Security. Package layout is **domain-driven**: `com.robogebra.cms.domain.<feature>.{controller, service, repository, model}` with 60+ sub-domains.

Q1. Explain IoC and Dependency Injection. How do you inject in this project?

Answer. Inversion of Control = the framework creates and wires objects, not me. **Dependency Injection** is how it hands collaborators in. I use **constructor injection with `final` fields** — the recommended style: dependencies are explicit, immutable, and the class is easy to unit-test (just `new` it with mocks). No field `@Autowired`.

In Robogebra. Lombok `@AllArgsConstructor` generates the constructor over the `final` fields, so Spring injects them (`SolutionVisibilityService`):

```
@Service
@AllArgsConstructor
@Transactional
public class SolutionVisibilityService {
    private final SolutionVisibilityPersistenceService solutionVisibilityPersistenceService;
    private final StudentEnrollmentService studentEnrollmentService;
    private final ExerciseItemService exerciseItemService;
    private final ExerciseService exerciseService;
    private final ChapterService chapterService;
}
```

Follow-up: constructor vs field injection? Constructor injection makes dependencies mandatory + immutable (`final`), fails fast if a bean is missing, and needs no Spring to test. Field injection hides dependencies and can't be `final`. That's why the whole codebase uses constructor injection.

Follow-up: bean scopes? Default **singleton** (one per context) — fine because our services are stateless. `prototype` / `request` / `session` exist for stateful cases.

Q2. What are the stereotype annotations and how is your app layered?

Answer. `@Component` is the base; `@Service`, `@Repository`, `@Controller` / `@RestController` are specializations that also convey intent (and `@Repository` adds persistence-exception translation). My layering is **Controller** → **Service** → **Persistence/Repository** → **Entity**, with DTOs at the edge.

In Robogebra. - `@RestController` `SolutionVisibilityController` — HTTP only, no logic. - `@Service` `SolutionVisibilityService` — business logic. - `@Service` `SolutionVisibilityPersistenceService` — thin wrapper over the repo (isolates DB access). - `@Repository` `SolutionVisibilityRepository` extends `MongoRepository` — data access.

That extra **persistence-service** layer between service and repository is a deliberate choice: it keeps all Mongo calls in one place and lets the business service stay focused.

Q3. How do you build REST APIs? Show a controller.

Answer. `@RestController` + `@RequestMapping` base path; method-level `@GetMapping/@PostMapping/@PatchMapping`; `@PathVariable` for URL params, `@RequestBody` + `@Valid` for the payload, and `@RequestAttribute` to read the authenticated user that the security filter placed on the request. I return DTOs, never entities.

In Robogebra (`SolutionVisibilityController`):

```
@AllArgsConstructor
@RestController
@RequestMapping("/api/solution-visibility")
public class SolutionVisibilityController {

    @PatchMapping("/status")
    public List<SolutionVisibilityDTO> updateStatus(
        @RequestAttribute(ApplicationConstants.INSTITUTE_USER_ATTRIBUTE) InstituteUser
        instituteUser,
        @Valid @RequestBody SolutionVisibilityUpdatedDTO request) {
        return service.updateStatus(instituteUser, request.getLevel(), request.getTargetId(),
            request.getKlassIds(), request.getStatus())
            .stream().map(SolutionVisibility::toDTO).toList();
    }

    @GetMapping("/exercise-items/{exerciseItemId}/solution-access")
    public StudentSolutionAccessDTO solutionAccess(
        @RequestAttribute(name = ApplicationConstants.INSTITUTE_USER_ATTRIBUTE, required =
        false) InstituteUser instituteUser,
        @PathVariable String exerciseItemId) {
        boolean allowed = Objects.isNull(instituteUser)
            || service.isSolutionEnabledForStudent(instituteUser, exerciseItemId);
        return
        StudentSolutionAccessDTO.builder().exerciseItemId(exerciseItemId).accessAllowed(allowed).build();
    }
}
```

Note `required = false` on the second `@RequestAttribute` — anonymous users have no `InstituteUser`, so the endpoint degrades gracefully instead of failing.

Q4. How do you handle exceptions globally?

Answer. A custom unchecked exception carrying an HTTP status + message, plus a `@ControllerAdvice` with `@ExceptionHandler`s that turn exceptions into a consistent JSON error body. Specific handlers first, a catch-all `ExceptionHandler` last. Business code just `throw`s; the advice formats + logs.

In Robogebra. Custom exception:

```
@Getter
public class APIException extends RuntimeException {
    private final HttpStatus status;
    private final String errorMessage;
    public APIException(HttpStatus status, String errorMessage) {
        super(errorMessage); this.status = status; this.errorMessage = errorMessage;
    }
}
```

Global advice (`ExceptionHandlerAdvice`):

```
@ControllerAdvice
@Slf4j
public class ExceptionControllerAdvice {
    @ExceptionHandler(MethodArgumentNotValidException.class) // @Valid failures → 400
    public ResponseEntity<Object> handleValidation(MethodArgumentNotValidException ex,
        WebRequest req) { ... }

    @ExceptionHandler(APIException.class) // our business errors → its
    status
    public ResponseEntity<ErrorMessageModel> handleAPIException(APIException ex, WebRequest
    req) {
        return ResponseEntity.status(ex.getStatus()).body(createErrorMessage(ex.getStatus(),
        ex.getMessage(), req));
    }

    @ExceptionHandler(Exception.class) // catch-all → 500 (no stack
    leak)
    public ResponseEntity<ErrorMessageModel> global(Exception ex, WebRequest req) { ... }
}
```

This is exactly how the "email already exists" error surfaces: the service throws `new APIException(HttpStatus.BAD_REQUEST, "An account with this email already exists.")`, the advice catches it and returns a clean `{status, message, timestamp, path}`.

Q5. How does validation work?

Answer. Bean Validation (Jakarta) — annotate DTO fields (`@NotNull`, `@NotEmpty`, `@Email` ...) with custom messages, then `@Valid` on the `@RequestBody` triggers it. Failures throw `MethodArgumentNotValidException`, which my advice maps to a 400 with the field message.

In Robogebra (`SolutionVisibilityUpdatedDTO`):

```
public class SolutionVisibilityUpdatedDTO {
    @NotNull(message = "level is required") private SolutionVisibilityLevel level;
    @NotNull(message = "targetId is required") private String targetId;
    @NotEmpty(message = "at least one klassId is required") private List<String> klassIds;
    @NotNull(message = "status is required") private SolutionVisibilityStatus
    status;
}
```

```
public ... updateStatus(@Valid @RequestBody SolutionVisibilityUpdatedDTO request) { ... }
```

Q6. How do you manage transactions?

Answer. `@Transactional` — declarative, AOP-proxy-based. Class-level for write services so each public method runs in a transaction (commit on success, rollback on runtime exception). `@Transactional(readOnly = true)` on query methods as an optimization hint.

In Robogebra. Class-level on write services:

```
@Service @AllArgsConstructor @Transactional
public class SolutionVisibilityService { ... } // updateStatus() is transactional
```

Read-only on a query (`RefreshTokenService`):

```
@Transactional(readOnly = true)
public String getRefreshToken(String userId) {
    return refreshTokenRepository.findById(userId).map(...).orElse(null);
}
```

Note (MongoDB caveat): Mongo multi-document transactions need a replica set. `@Transactional` still gives a consistent boundary; for most single-document writes Mongo is atomic per-document anyway.

Q7. Explain Spring Boot auto-configuration & `@SpringBootApplication` .

Answer. `@SpringBootApplication` = `@Configuration` + `@ComponentScan` + `@EnableAutoConfiguration` . Auto-config inspects the classpath and properties and wires sensible beans — e.g. seeing Spring Data MongoDB + a `spring.data.mongodb.uri` auto-creates the `MongoTemplate` /repositories. I override with my own `@Beans` or `@ConditionalOnProperty` when needed.

In Robogebra — a conditional bean (`CacheConfig`) that swaps the cache manager based on a property:

```
@Configuration @EnableCaching
public class CacheConfig {
    @Bean @ConditionalOnProperty(prefix = "cache", name = "enabled", havingValue = "true")
    public CacheManager cacheManager() { /* Caffeine caches */ }

    @Bean @ConditionalOnProperty(prefix = "cache", name = "enabled", havingValue = "false")
    public CacheManager noOpCacheManager() { return new NoOpCacheManager(); }
}
```

Q8. How is configuration / profiles handled?

Answer. `application.yml` with environment-variable placeholders and defaults, `@Value` / `@ConfigurationProperties` to bind, and profile files (`application-dev.yml`) for per-env values.

In Robogebra (`application.yml`) — Mongo URI, OAuth2, JWT, env overrides:

```
spring:
  data:
    mongodb: { uri: "mongodb://localhost:27017/robogebra_cms_db", auto-index-creation: true }
  security:
    oauth2:
      client:
        registration:
          google: { client-id: ${GOOGLE_SSO_CLIENT_ID}, client-secret: ${GOOGLE_SSO_SECRET} }
  auth:
    jwt: { secret: ${AUTH_JWT_SECRET:robogebra-sso-secret-2026}, expiry: 86400000, issuer: ${ISSUER:robogebra-auth} }
```

`${VAR:default}` = read env var, fall back to default — the pattern that keeps secrets out of the repo.

Q9. @Async and scheduled jobs?

Answer. `@Async` runs a method on a thread pool (fire-and-forget) — good for I/O side-effects that shouldn't block the request. For scheduled work we use **Quartz Jobs** (and Spring's `@Scheduled` cron elsewhere).

In Robogebra. Notifications are async so signup/payment responses aren't blocked (`NotificationServiceFactory`):

```
@Async public void sendWelcomeNotification(User user)
{ getNotificationService(user.getUserName())....; }
@Async public void sendOtp(String userName, String otp)
{ getNotificationService(userName).sendOTP(userName, otp); }
```

Scheduled subscription activation (`InstituteSubscriptionActivationJob` implements `org.quartz.Job`):

```
@Override public void execute(JobExecutionContext ctx) {
    instituteSubscriptionSchedulerService.handleUpdateScheduledToActiveSubscriptions();
}
```

That factory method also shows the **Factory pattern** — it returns WhatsApp vs Email notification service based on whether the username is a phone number.

Q10. What design patterns show up in the codebase?

Answer. - **Builder** — Lombok `@Builder` on entities/DTOs/models (fluent, immutable-ish construction). - **Repository** — `MongoRepository` interfaces abstract data access. - **Service layer** — orchestration between controllers and repos. - **DTO** — decouple API from persistence. - **Factory** — `NotificationServiceFactory.getNotificationService(username)` picks the channel. - **Template method** — `AbstractServiceAuthenticationFilter` (base algorithm + overridable hooks). - **Strategy** — pluggable auth services resolved per username/token type.

Rapid-fire recap

Topic	Robogebra example	File
DI / IoC	<code>@AllArgsConstructor</code> + <code>final</code>	<code>SolutionVisibilityService.java</code>
REST	<code>@RestController</code> , <code>@PathVariable</code> , <code>@RequestAttribute</code>	<code>SolutionVisibilityController.java</code>
Exceptions	<code>APIException</code> + <code>@ControllerAdvice</code>	<code>ExceptionHandlerAdvice.java</code>
Validation	<code>@NotNull</code> / <code>@NotEmpty</code> + <code>@Valid</code>	<code>SolutionVisibilityUpdateDTO.java</code>
Transactions	<code>@Transactional</code> , <code>readOnly</code>	services
Auto-config	<code>@Bean</code> + <code>@ConditionalOnProperty</code>	<code>CacheConfig.java</code>
Config	<code>\${VAR:default}</code> in yml	<code>application.yml</code>
Async/ Sched	<code>@Async</code> , Quartz <code>Job</code>	<code>NotificationServiceFactory.java</code>
Patterns	Builder/Repository/Factory/Template	across domains

5. Database (MongoDB) — Interview Q&A tied to Robogebra

Stack: MongoDB via Spring Data MongoDB. Portal DB `robogebra_cms_db` ; CRM has its own `robogebra-coupon-mgmt` (database-per-service). No SQL/JPA in the core learning platform.

Q1. Why MongoDB (document DB) over a relational DB here?

Answer. Our core domain is **content** — books → chapters → exercises → items → solutions → step-by-step details. It's naturally hierarchical, deeply nested, and varies by exercise **type** (numeric vs algebra vs MCQ have different shapes). A document model fits that flexible, evolving schema far better than rigid tables + many joins. We also want horizontal scaling and fast reads of whole content trees. Where we need relational-style integrity (payments, coupons) that lives in the **CRM** service with its own DB.

Trade-off I'd volunteer: Mongo trades multi-table ACID/joins for flexibility. We accept eventual consistency for analytics, and use **synchronous REST + careful modeling** for the money paths. If the domain were transaction-heavy and highly relational, I'd pick Postgres.

Q2. How do you model documents and relationships? `@DBRef` ?

Answer. `@Document` maps a class to a collection. For relationships I choose **embed vs reference**: embed when the child is owned and read together; **reference** (`@DBRef`, storing the related id) when entities are large, shared, or queried independently.

In Robogebra (`exercise/repository/model/ExerciseEntity.java`) — an exercise **references** its book and chapter:

```
@Document(collection = "exercise")
@CompoundIndexes({
    @CompoundIndex(name = "exercise_name_index_book_id_chapter_id",
        def = "{ 'name':1, 'index':1, 'book.id':1, 'chapter.id':1 }", unique = true),
    @CompoundIndex(name = "chapter_id_order_index", def = "{ 'chapter.$id':1, 'order':1 }")
})
public class ExerciseEntity {
    @Id private String id;
    @Indexed @DBRef @Field("book")    private BookEntity book;
    @Indexed @DBRef @Field("chapter") private ChapterEntity chapter;
}
```

Book/Chapter are referenced (not embedded) because they're large and shared across many exercises.

Follow-up: `@DBRef` **downsides?** It doesn't join server-side — Spring resolves it with extra queries, so overusing it causes N+1. For hot read paths I'd store just the id or use an `$lookup` aggregation instead.

Q3. How do repositories and queries work in Spring Data Mongo?

Answer. Extend `MongoRepository<Entity, IdType>` — you get CRUD free. Add **derived query methods** (Spring parses the method name into a query), or `@Aggregation` for pipelines, or `@Query` for raw filters.

In Robogebra. Derived queries (`SolutionVisibilityRepository`):

```
public interface SolutionVisibilityRepository extends
MongoRepository<SolutionVisibilityEntity, String> {
    Optional<SolutionVisibilityEntity> findByKlassIdAndTargetId(String klassId, String
targetId);
    List<SolutionVisibilityEntity> findByKlassIdInAndTargetIdIn(List<String> klassIds,
List<String> targetIds);
}
```

Aggregation pipeline (`ExerciseRepository`) — match + sort in the DB:

```
@Aggregation(pipeline = {
    "{ '$match': { 'chapter.$id': ObjectId(??) } }",
    "{ '$sort': { 'order': 1 } }"
})
List<ExerciseEntity> findAllByChapterId(String chapterId);
```

`findByKlassIdInAndTargetIdIn` is the one that powers the whole visibility check — it fetches all candidate rows for many classes and many targets in **one** round trip.

Q4. Explain indexing in your schema and why it matters.

Answer. Indexes turn collection scans into fast lookups. I use `@Indexed` on frequently-filtered fields and `@CompoundIndex` for multi-field queries and **uniqueness constraints**. The compound index order should match query + sort order.

In Robogebra. The visibility collection has a **unique compound index** (`klass_id, target_id`) — this both speeds lookups *and* enforces "one rule per class per target" at the DB level (so enable/disable is an upsert, never a duplicate):

```
@CompoundIndex(name = "klass_id_target_id_idx", def = "{ 'klass_id':1, 'target_id':1}", unique =
true)
```

Exercises index (`chapter.$id, order`) so "get chapter's exercises sorted" is index-backed. We enable `auto-index-creation: true` in dev.

Q5. Show a real query-design decision you made.

Answer. The **solution-visibility precedence check** is my favorite DB story. A student's access to a solution depends on the most-specific rule among *item* → *exercise* → *chapter*, across *all* their classes. Naively that's several queries and nested loops. Instead:

1. Build the precedence id list once: `List.of(exerciseItemId, exerciseId, chapterId)` .
2. **One** query for all classes × all three targets: `findByKlassIdInAndTargetIdIn(klassIds, precedence)` .
3. `groupingBy(targetId)` into a `Map` in memory.
4. Walk precedence most-specific-first; first level that has a rule decides (`anyMatch ENABLED`); default open.

```

Map<String, List<SolutionVisibilityEntity>> byTarget =
    persistence.findByClassIdsAndTargetIds(klassIds, precedence)
        .stream().collect(Collectors.groupingBy(SolutionVisibilityEntity::getTargetId));
for (String targetId : precedence) {
    var rows = byTarget.get(targetId);
    if (rows != null && !rows.isEmpty())
        return rows.stream().anyMatch(r -> r.getStatus() == SolutionVisibilityStatus.ENABLED);
}
return true;

```

One DB round trip, O(1) lookups, correct override semantics. That's a concrete "how I think about queries + data structures together" answer.

Q6. Audit fields, soft deletes, timestamps?

Answer. Spring Data auditing (`@CreatedDate` , `@LastModifiedDate`) auto-stamps documents; soft-delete flags instead of hard deletes where history matters.

In Robogebra. Entities carry `@CreatedDate` / `@LastModifiedDate` (`created_at` / `updated_at`) managed automatically, and the visibility model uses an **upsert** (`findByClassIdAndTargetId(...).map(update).orElseGet(create)`) so a teacher toggling a class just flips the existing row's status.

Q7. MongoDB vs MySQL — the classic comparison.

Answer.

	MongoDB (us)	MySQL
Model	Documents (JSON/BSON), flexible schema	Tables, fixed schema
Relations	Embed / <code>@DBRef</code> / <code>\$lookup</code>	Foreign keys + joins
Transactions	Per-doc atomic; multi-doc needs replica set	Full ACID
Scaling	Horizontal (sharding)	Vertical primarily
Fits	Nested, evolving content	Highly relational, transactional

We use Mongo because content is hierarchical and schema-flexible; a payments/coupons system would be a fair place for SQL (and the CRM keeps that concern separate).

Rapid-fire recap

Topic	Robogeбра example	File
Document + refs	<code>@Document</code> , <code>@DBRef</code> book/chapter	<code>ExerciseEntity.java</code>
Repositories	derived + <code>@Aggregation</code>	<code>SolutionVisibilityRepository.java</code> , <code>ExerciseRepository.java</code>
Indexing	unique compound <code>(klass_id,target_id)</code>	<code>SolutionVisibilityEntity.java</code>
Query design	1 query + <code>groupBy</code> + precedence	<code>SolutionVisibilityService.java</code>
Auditing/ upsert	<code>@CreateDate</code> , find-or-create	visibility model

6. Microservices & Architecture — Q&A tied to Robogeбра

Reality: a **microservices hybrid** — a monolithic learning core (portal) plus independent services (CRM, admin/AI) and several separate frontends. Be honest that it's *pragmatic hybrid*, not textbook pure microservices — that's the senior answer.

Q1. Describe the overall architecture of Robogeбра.

Answer. Three backend services and multiple frontends, each owning a business domain:

- **robogeбра-portal** (Spring Boot 3.2, port 3000) — the monolithic core: all student/teacher learning flows, content, solutions, progress, payments-entry. MongoDB `robogeбра_cms_db`.
- **robogeбра-admin-portal** (Spring Boot, port 4000) — content authoring + **AI generation** (calls Python ML services). Separated so heavy content pipelines don't affect the student-facing API.
- **robogeбра-crm** (Node.js/Express, port 3001) — B2B: institutes, classes, students, coupons, subscriptions. Own DB `robogeбра-coupon-mgmt`.
- **Frontends:** `robogeбра-web` (Angular) & `robogeбра-mobile` (Ionic) for students/teachers; `robogeбра-backoffice` (Angular) for ops; `robogeбра-crm-web` (React) for CRM admins; `robogeбра-website` (static).

One-line pitch: "Robogeбра is a modular microservices system for a two-sided market — students and institutions. The portal is our monolithic learning core; the CRM is an independent service for B2B; AI/content is isolated in the admin-portal. Each frontend is deployed independently. It's a pragmatic hybrid that balances startup speed with the ability to scale the busy parts separately."

Q2. Is this "real" microservices? Monolith or microservices?

Answer (honest, senior). It's a **hybrid**: the portal is a **modular monolith** (one deployable, internally split into 60+ domains), while CRM and admin-portal are **separate deployable services with their own databases**. So we get microservice benefits (independent scaling/deploy of CRM & AI, DB-per-service, tech-fit — Node for CRM, Spring for core) without the full operational cost of decomposing the learning core prematurely. I'd only split the portal further when a specific domain needs independent scaling or team ownership — not for its own sake.

Q3. How do the services communicate?

Answer. Mostly **synchronous REST** over HTTPS, with JWT/service tokens; cron for time-based workflows.

- **Portal** → **CRM** (REST/WebClient): coupon validation & redemption, subscription/institute checks. e.g. portal calls CRM's `/v1/coupons/redeem` during payment.
- **Portal** → **Admin-portal / Python AI**: solution generation, interactive content — REST calls to ML endpoints.
- **Frontends** → **backends**: JWT bearer token on every request.

- **Service-to-service auth:** portal exposes `@BypassJwtAuth` endpoints for trusted callers (webhooks, CRM) that use a service token instead of a user JWT.
- **Time-based:** Quartz/cron jobs (e.g. nightly subscription activation ~12:05 AM IST).

Trade-off: synchronous REST is simple and consistent but couples availability. For non-critical events (analytics, notifications) I'd move to async/queue; critical money paths stay synchronous for immediate consistency.

Q4. How is authentication handled across services? (also see 07-aws.md)

Answer. AWS Cognito (three user pools: `user`, `email-user`, `institute-user`) authenticates identity; the **portal issues/validates JWTs** and enforces permissions. Frontends store the JWT and send it as `Authorization: Bearer`. The portal's `JwtAuthenticationFilter` validates (SSO JWT or Cognito), sets the security context, and puts the resolved user on the request. OAuth2 SSO (Google/LinkedIn) is supported for frictionless onboarding.

Three pools = clean **multi-tenant** isolation: B2C consumers vs email signups vs institute (B2B) admins/teachers each have their own namespace.

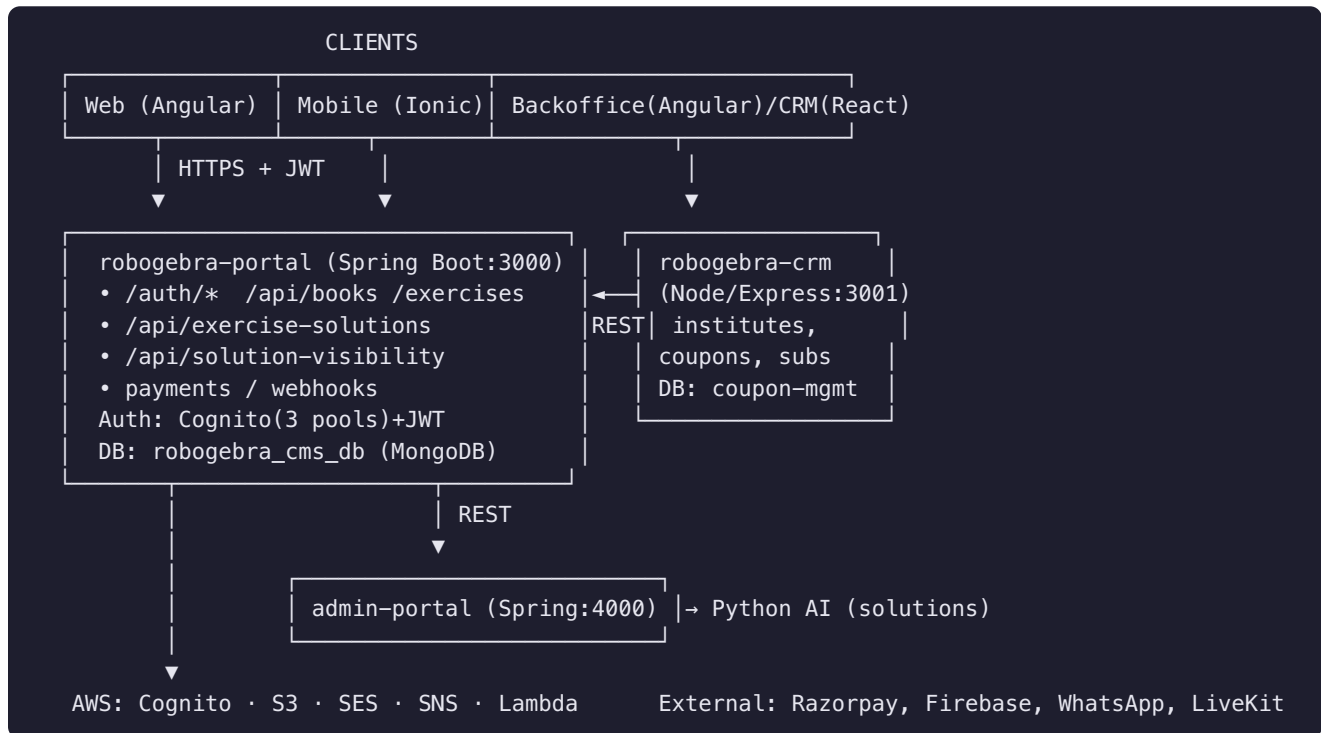
Q5. How do you handle data consistency across services?

Answer. Database-per-service (portal's CMS DB vs CRM's coupon-mgmt DB) avoids accidental coupling but means no cross-service transaction. So: keep each write inside one service's DB (atomic there), use **synchronous REST** for the few cross-service critical actions (coupon redemption during payment), accept **eventual consistency** for analytics, and use idempotent operations + retries. For true cross-service workflows I'd use the **saga** pattern (compensating actions) rather than a distributed transaction.

Q6. How would you scale and make this resilient?

Answer. - Stateless services → horizontal scaling behind a load balancer / K8s HPA. - **Caching** — Caffeine in-process for rarely-changing content (books/chapters/exercises), with property-toggle + TTLs. - **MongoDB** — indexes for high-cardinality queries; sharding if needed. - **CDN + S3** for media/assets. - **Resilience** — timeouts + retries + circuit breakers on outbound calls (portal→CRM, portal→AI); AI failure should degrade gracefully (cached/basic answer), not break the student flow. - **Observability** — logs, PostHog product analytics (prod).

Q7. Whiteboard the system.



Q8. Example end-to-end flow (student pays with a coupon).

Answer. Frontend opens Razorpay → gets `payment_id` → `POST /api/payment` to portal → portal verifies with Razorpay → **portal calls CRM** `/v1/coupons/redeem` to validate/redeem the coupon → portal writes `Payment` + `Subscription` → a nightly cron activates the subscription. This is the concrete "services talk over REST, each owns its data" story.

Rapid-fire recap

Topic	Robogebra reality
Style	Microservices hybrid (modular-monolith core + separate CRM/AI)
Services	portal (Spring:3000), admin-portal (Spring:4000), CRM (Node:3001)
Comms	synchronous REST + JWT/service token; cron for scheduled
Auth	Cognito 3 pools + portal-issued JWT + OAuth2
Data	DB-per-service; eventual consistency + REST for critical paths
Scale	stateless + Caffeine cache + Mongo indexes + CDN + circuit breakers

7. AWS & Cloud Services — Interview Q&A tied to Robogebra

Services used: Cognito (auth), S3 (assets), SES (email), SNS (push), Lambda (serverless content), plus Firebase for real-time/push. Region: `ap-south-1` (Mumbai).

Q1. How does authentication use AWS Cognito? Why three user pools?

Answer. **Cognito** is a managed identity provider — it handles signup, OTP/MFA, password policies, and token issuance, so we don't build/secure that ourselves. We run **three separate user pools** for clean multi-tenant isolation:

Pool	Segment	Why separate
<code>user</code>	B2C individual students/teachers	consumer signups
<code>email-user</code>	email-based signups	alternate login flow
<code>institute-user</code>	B2B institute admins/teachers	school/coaching namespace, different policies

Flow: frontend → portal `/auth/*` → portal validates credentials against the right Cognito pool → portal issues its **own JWT** (custom claims: `userId`, `role`, `permissions`) → frontend stores it → subsequent requests carry `Authorization: Bearer` → portal's `JwtAuthenticationFilter` validates (Cognito or our SSO JWT). OAuth2 SSO (Google/LinkedIn) is wired via Spring Security for one-click onboarding.

Why separate pools instead of one with groups? Different segments have different password/MFA policies, signup flows, and blast-radius isolation — a config change or breach in one pool can't affect another. That's the senior justification.

Q2. Where does S3 fit, and how do you serve files securely?

Answer. **S3** stores user-generated assets and published textbook content (buckets `robogebra-dev-assets`, `robogebra-core-content`). We never make objects public — the backend generates **pre-signed URLs** with a short expiry (5 min) so the client can fetch/upload directly to S3 without the file passing through our servers. `dev` / `prod` path prefixes separate environments; a CDN fronts assets for delivery.

Classic interview design (upload flow): client asks portal for a **pre-signed PUT URL** → uploads the file straight to S3 → stores only the **object key/metadata** in MongoDB → later reads use a pre-signed **GET** URL. Keeps large binaries out of the app tier and out of the DB.

Q3. What other AWS services do you use and why?

Answer. - **SES** — transactional email (OTP, payment receipts, notifications), sender `noreply@robogebra.ai`. - **SNS** — push notifications / alerts. - **Lambda** — serverless functions for content processing (interactive answer/graph generation, semantic search, coupon validation) — event-driven, scales to zero, isolates spiky ML-ish work from the main API. - **Firebase** — real-time data + cloud messaging (complements SNS for device push).

The theme: **offload cross-cutting/spiky work to managed services** so the portal stays focused and responsive.

Q4. Tie it together: the signup + auth journey on AWS.

Answer. 1. User submits signup (email/phone) → portal `/auth/create-user`. 2. Portal calls **Cognito** (correct pool) to create the user + trigger **OTP** (delivered via SES email / WhatsApp). 3. Duplicate? Portal's find-or-create throws `APIException(400, "An account with this email already exists.")` (see `02-backend.md` / `04-spring.md`), unless it's a valid **link-Cognito-to-existing-SSO-user** case. 4. User verifies OTP → Cognito confirms → portal issues its **JWT**. 5. Every later request: `Authorization: Bearer <JWT>` → `JwtAuthenticationFilter` validates (Cognito SDK or SSO issuer check) → sets security context + request attributes → controller reads the user via `@RequestAttribute`. 6. Cognito SDK errors are mapped to HTTP statuses (`NotAuthorizedException` → 401, `TooManyRequestsException` → 400) so failures are typed, not 500s.

Q5. Config & secrets across environments?

Answer. Config lives in `application.yml` with **environment-variable placeholders + defaults** (`${VAR:default}`); real secrets (Cognito client secrets, JWT secret, AWS keys, OAuth client secrets) come from the environment, never committed. Region pinned to `ap-south-1`. This keeps the same artifact deployable across dev/prod with different env values.

```
aws:
  cognito:
    user: { region: ap-south-1, userPoolId: ..., clientId: ...,
clientSecretId: ... }
    institute-user: { region: ap-south-1, userPoolId: ..., clientId: ...,
clientSecretId: ... }
  auth:
    jwt: { secret: ${AUTH_JWT_SECRET:...}, expiry: 86400000, issuer: ${ISSUER:robogebra-auth} }
```

Q6. If asked about scaling/resilience on AWS.

Answer. Stateless services scale horizontally (containers/K8s or ECS); S3 + CloudFront for static/media; Lambda auto-scales for bursty content jobs; Cognito & SES are managed (no ops burden). Outbound calls (to CRM/AI) get timeouts + retries + circuit breakers; a failing AI/Lambda path degrades gracefully rather than breaking the student flow. Multi-AZ Mongo + indexes for data-tier resilience.

Rapid-fire recap

Service	Use in Robogebra
Cognito	Auth — 3 pools (user / email-user / institute-user) + OAuth2 SSO
S3	Assets & textbook content; pre-signed URLs; dev/prod prefixes
SES	Transactional email (OTP, receipts)
SNS	Push notifications
Lambda	Serverless content processing (interactive answer/graph, search, coupon)
Firebase	Real-time + device push
Secrets	<code>\${ENV:default}</code> in <code>application.yml</code> , region <code>ap-south-1</code>

8. Error Handling (Deep Dive) — tied to Robogebra

How the portal turns *any* failure — validation, business, DB, external service, auth — into a **consistent JSON error** the frontend can trust, without leaking internals.

Q1. Walk me through your end-to-end error-handling strategy.

Answer. Three principles: (1) **throw typed exceptions** from business code (never return error codes), (2) **one place formats them** — a `@ControllerAdvice` — so every error has the same JSON shape, and (3) **log full detail server-side, return a safe message client-side**. Two delivery paths: normal request errors go through the advice; errors thrown **inside security filters** (before the advice can run) are written directly by an `ExceptionHandlerWriter`.

The response shape every client gets (`ErrorMessageModel`):

```
@Data @Builder
public class ErrorMessageModel {
    private Integer statusCode; // HTTP status
    private Date    timestamp;
    private String  message;    // user-facing
    private String  description; // request path
    private Integer errorCode;  // optional Robogebra code for programmatic handling
}
```

Q2. Show the global exception handler. How is it organized?

Answer. A `@ControllerAdvice` with ~2 dozen `@ExceptionHandler` methods, ordered **specific** → **generic**, ending in a catch-all `Exception` that returns 500 with a generic message (so stack traces never reach the client).

In Robogebra (`common/exception/handler/ExceptionHandlerControllerAdvice.java`):

```

@ControllerAdvice @Slf4j
public class ExceptionControllerAdvice {

    @ExceptionHandler(MethodArgumentNotValidException.class)    // @Valid body failures → 400
    public ResponseEntity<Object> handleValidation(MethodArgumentNotValidException ex,
WebRequest req) {
        String msg = findErrorMessageForValidation(ex);
        return ResponseEntity.status(BAD_REQUEST).body(createErrorMessage(BAD_REQUEST, msg,
req));
    }

    @ExceptionHandler(APIException.class)                      // our business errors → its own
status
    public ResponseEntity<ErrorMessageModel> handleAPI(APIException ex, WebRequest req) {
        log.error(ex.getMessage(), ex);
        return ResponseEntity.status(ex.getStatus()).body(createErrorMessage(ex.getStatus(),
ex.getMessage(), req));
    }

    @ExceptionHandler(Exception.class)                          // catch-all → 500, generic
message
    public ResponseEntity<ErrorMessageModel> global(Exception ex, WebRequest req) {
        log.error("Unhandled exception: ", ex);                    // full stack to server logs only
        return ResponseEntity.status(INTERNAL_SERVER_ERROR)
            .body(createErrorMessage(INTERNAL_SERVER_ERROR, "Something went wrong. Please try
again later.", req));
    }
}

```

Coverage (know a handful, not all): validation (`MethodArgumentNotValidException` , `ConstraintViolationException` , `MethodArgumentTypeMismatchException`), resources (`ResourceNotFoundException` →404, `ResourceConflictException` →409), security (`SecurityPermissionException` →403), infra (`MongoWriteException` , `DatabaseException` , `MaxUploadSizeExceededException`), external (`PaymentException` , `ExternalApiException` , `CouponValidationException`), auth (`CognitoException` , `SsoTokenException`), and the generic `Exception` .

Q3. What custom exceptions do you have, and how are they designed?

Answer. All extend `RuntimeException` (unchecked — so I don't pollute signatures). Three shapes by need:

1. **Status + message** — the workhorse: `APIException` , `ExternalApiException` , `DatabaseException` , `PaymentException` carry an `HttpStatus` + message.
2. **Message-only** — `ResourceNotFoundException("Exercise item", id)` , `ResourceConflictException` , `SecurityPermissionException` .
3. **Error-code-based** — `CognitoException` , `SsoTokenException` , `UsageLimitExceedException` carry a `robogebraErrorCode` (from an `ErrorCode` enum) so the client can branch programmatically (e.g., "token expired → silent refresh").

```
@Getter
public class APIException extends RuntimeException {
    private final HttpStatus status;
    private final String errorMessage;
    public APIException(HttpStatus status, String msg) { super(msg); this.status = status;
    this.errorMessage = msg; }
}
```

Why unchecked? Business code stays clean (`throw new APIException(...)`), the advice is the single catch point, and I don't force every caller to declare/handle checked exceptions.

Q4. How do you map an *error type* to an HTTP status cleanly?

Answer. With an **enum + switch expression** — no scattered `if/else`. Two patterns:

Domain error → **status** (`CouponValidationException`):

```
HttpStatus status = switch (ex.getErrorType()) {
    case INVALID_COUPON                                -> HttpStatus.BAD_REQUEST;
    case SERVICE_UNAVAILABLE, LAMBDA_ERROR, NETWORK_ERROR -> HttpStatus.SERVICE_UNAVAILABLE;
    case PARSING_ERROR                                  -> HttpStatus.INTERNAL_SERVER_ERROR;
};
```

Error-code registry (`ErrorCode` enum) — one place mapping a code to a status, reused by many exceptions:

```
@Getter @AllArgsConstructor
public enum ErrorCode {
    COGNITO_TOKEN_EXPIRED(30000, HttpStatus.UNAUTHORIZED),
    SOLUTION_USAGE_LIMIT_EXCEEDED(50001, HttpStatus.FORBIDDEN),
    MULTI_USER_LOGIN_EXCEEDED(60001, HttpStatus.UNAUTHORIZED);
    private final Integer code; private final HttpStatus httpStatus;
}
```

The numeric `errorCode` is the real senior touch: the frontend can react to `30000` (token expired) differently from `50001` (upgrade prompt) without string-matching messages.

Q5. How do you translate messy third-party errors into clean user messages?

Answer. External SDKs throw verbose, technical exceptions. I **catch at the boundary** and remap: AWS Cognito exceptions → friendly copy via a lookup enum; WebClient failures → a normalized `ExternalApiException` with the right status.

Cognito → **user message** (`CognitoExceptionType`):

```
public enum CognitoExceptionType {
    USERNAME_EXISTS("UsernameExistsException", "Phone number already registered"),
    NOT_AUTHORIZED("NotAuthorizedException", "Incorrect password"),
    CODE_MISMATCH("CodeMismatchException", "Verification code incorrect. Please try again"),
    TOO_MANY_REQUESTS("TooManyRequestsException", "Too many requests. Please try again
later");
    // getCustomMessage(raw) scans for the AWS name and returns the friendly text
}
```

WebClient boundary mapping (`SyncApiClientService`) — distinguish *downstream* 4xx/5xx from *network/timeout*:

```
catch (WebClientResponseException e) { // downstream returned an error
    throw new ExternalApiException(INTERNAL_SERVER_ERROR, e.getResponseBodyAsString());
} catch (WebClientRequestException e) { // timeout / connection refused / DNS
    throw new ExternalApiException(GATEWAY_TIMEOUT, "Portal API timed out");
}
```

MongoDB duplicate key → **readable message** — regex-parse the driver error:

```
Pattern.compile("dup key: \\{ (.+?): \\\"(.+?)\\\" }"); // → "The email 'a@b.com' already
exists."
```

Q6. Errors thrown in a filter can't be caught by `@ControllerAdvice`. How do you handle those?

Answer. Correct — the advice only wraps controller invocations; a `OncePerRequestFilter` runs *before* that. So auth filters catch their own exceptions and write the **same** `ErrorMessageModel` JSON directly via a shared `ExceptionResponseWriter`, keeping the contract identical.

In Robogebra (`JwtAuthenticationFilter` → `ExceptionResponseWriter`):

```
// in the filter
catch (CognitoException e) { responseWriter.writeCognitoException(response, request, e); }
catch (SsoTokenException e) { responseWriter.writeSsoTokenException(response, request, e); }
```

```
// ExceptionResponseWriter — builds the SAME ErrorMessageModel and writes JSON to the response
response.setStatus(ex.getStatus()); response.setContentType("application/json");
objectMapper.writeValue(response.getOutputStream(), ErrorMessageModel.builder()
    .errorCode(ex.getRobogebraErrorCode()).statusCode(ex.getStatus())
    .message(CognitoExceptionType.getCustomMessage(ex.getErrorMessage()))
    .description(request.getRequestURI()).timestamp(DateUtils.now()).build());
```

This is a great "I understand the framework internals" point — most candidates don't realize the advice can't see filter exceptions.

Q7. Logging & security — what does the client see vs the server?

Answer. `@Slf4j` everywhere; `log.error("...", ex)` logs the **full stack trace to server logs** for debugging. The **client only gets a safe message** — the generic 500 handler returns *"Something went wrong. Please try again later."*, never the exception detail. That prevents leaking stack traces, class names,

or query internals (an OWASP concern). Log levels: `error` for failures, `warn` for soft limits (e.g., session cap), `info` for successful sensitive ops (logout), `debug` for token/handler tracing.

Q8. How does this connect to the frontend? (full loop)

Answer. Backend throws → advice/writer builds `{statusCode, message, errorCode}` → Angular's HTTP layer `catchError`s → a central `NotificationService` extracts `error.message` and toasts it. Specific `errorCode`s drive behavior — e.g. a token-expired code triggers the interceptor's silent refresh instead of a toast. So the "email already exists" message the user sees is literally the backend's `APIException` message flowing through this pipeline (frontend side in `01-frontend.md` Q14).

Rapid-fire recap

Concept	Robogebra implementation	File
Central formatting	<code>@ControllerAdvice</code> , specific→generic	<code>ExceptionHandlerAdvice.java</code>
Response shape	<code>ErrorMessageModel {status,msg,code,ts,path}</code>	<code>ErrorMessageModel.java</code>
Custom exceptions	unchecked, status/message/errorCode	<code>APIException.java</code> , <code>CognitoException.java</code>
Type→status	switch expr + <code>ErrorCode</code> enum	<code>ErrorCode.java</code> , <code>CouponValidationException.java</code>
3rd-party remap	<code>CognitoExceptionType</code> , WebClient boundary	<code>CognitoExceptionType.java</code> , <code>SyncApiClientService.java</code>
Filter errors	<code>ExceptionResponseWriter</code> writes same JSON	<code>ExceptionResponseWriter.java</code>
Security	full stack to logs, generic msg to client	generic <code>Exception</code> handler

9. Authentication & Authorization (Deep Dive) — tied to Robogebra

The single most-probed senior area. Keep the distinction crisp: **Authentication** = **who are you** (JWT/Cognito, filter chain), **Authorization** = **what can you do** (`@PermissionContext` + role/permission model).

PART A — AUTHENTICATION ("who are you")

Q1. Describe your Spring Security setup.

Answer. Stateless, JWT-based. No server sessions, so **CSRF is disabled** and the session policy is `STATELESS`. CORS is configured via a bean. `/api/**` requires authentication; OAuth2/login/well-known endpoints are permitted. Auth happens in a **custom filter chain**, not the default username/password filter.

In Robogebra (`config/SecurityConfig.java`):

```
http.csrf(AbstractHttpConfigurer::disable)
    .cors(c -> c.configurationSource(corsConfigurationSource))
    .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(a -> a
        .requestMatchers("/.well-known/**", "/oauth2/**", "/login/**").permitAll()
        .requestMatchers("/api/**").authenticated()
        .anyRequest().permitAll()
        .oauth2Login(oauth -> oauth.successHandler(socialAuthSuccessHandler) ...);
```

Why stateless? Horizontal scaling — any instance can serve any request because identity travels in the token, not a sticky session. That's the microservices-friendly choice.

Q2. Explain your security filter chain and its order.

Answer. A pipeline of `OncePerRequestFilter` s, each with a job and an explicit `@Order` . Service-to-service auth runs **before** user JWT auth, so trusted callers short-circuit. `JwtAuthenticationFilter` (Order 6) is the main user authenticator.

Order	Filter	Authenticates
—	<code>FilterChainExceptionHandler</code>	catches filter exceptions (first)
2	<code>AnonymousContentAuthenticationFilter</code>	public/anonymous content
3	<code>CrmServiceAuthenticationFilter</code>	<code>CRM_SERVICE_ACCOUNT</code> (service token)
4	<code>InternalServiceAuthenticationFilter</code>	<code>INTERNAL_SERVICE_ACCOUNT</code>
5	<code>WebhookAuthorizationFilter</code>	webhooks (RevenueCat) — no SecurityContext
6	<code>JwtAuthenticationFilter</code>	regular users (JWT/Cognito)
8	<code>MultiLoginDetectionFilter</code>	concurrent-session enforcement
—	<code>BookAccessRestrictionFilter</code> , <code>FeatureUsageLimitFilter</code> , <code>PreferredLanguageFilter</code>	subscription/feature/lang

Q3. How does the JWT filter work, and how does the user reach the controller?

Answer. Extract `Bearer` token → pick validation path (our SSO JWT vs Cognito) → load user → set `SecurityContext` → **stash the user on the request as attributes** → continue. Controllers then read those attributes with `@RequestAttribute` — they never re-authenticate.

In Robogebra (`JwtAuthenticationFilter`):

```
String token = authService.getTokenFromRequest(req);
JwtAuth jwtAuth = authService.isSSOToken(token)
    ? authService.getAuthDetailsBySSOToken(token) // our HMAC JWT
    : authService.getAuthDetailsByToken(token);    // AWS Cognito (RSA/JWKS)

SecurityContextHolder.getContext().setAuthentication(
    new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities()));

req.setAttribute(LOGGED_USER_ATTRIBUTE, jwtAuth);
req.setAttribute(ACTIVE_USER_ID_ATTRIBUTE, jwtAuth.getId());
instituteUserService.findInstituteUser(userId).ifPresentOrElse(
    iu -> req.setAttribute(INSTITUTE_USER_ATTRIBUTE, iu),
    () -> req.setAttribute(INSTITUTE_USER_ATTRIBUTE, null)); // not an institute user
```

Controller consumes it:

```
public ... updateStatus(@RequestAttribute(INSTITUTE_USER_ATTRIBUTE) InstituteUser
    instituteUser, ...) { ... }
```

Q4. Two JWT types? Explain SSO JWT vs Cognito JWT.

Answer. We support both:

- **Our SSO JWT** (`JwtService`, JJWT) — signed **HMAC-SHA256** with a shared secret; issued after OAuth2/admin login; carries claims `userId`, `username`, `role` (+ institute claims for institute users).

- **AWS Cognito JWT** — signed **RSA256**; we validate the signature against Cognito's **JWKS** (`.well-known/jwks.json`), verifying issuer + expiry with a small clock-skew leeway.

SSO token creation (`JwtService`):

```
return Jwts.builder().setSubject(userId)
    .claim("username", username).claim("role", role)
    .setIssuer("robogebra-auth").setIssuedAt(new Date())
    .setExpiration(new Date(System.currentTimeMillis() + expiry))
    .signWith(getKey(), SignatureAlgorithm.HS256).compact();
```

Cognito validation (`CognitoJwtValidator`): decode → read `iss` → pick the right pool → fetch JWKS → build RSA verifier (`acceptLeeway(60)`) → verify → return the Cognito subject id → look up our `User` by `cognitoId`.

Why HMAC vs RSA? HMAC (symmetric) is fine for tokens we issue and verify. Cognito uses RSA (asymmetric) so anyone can verify with the public JWKS without holding a secret — which is why we validate its tokens against the published key set.

Q5. Why three Cognito user pools?

Answer. Multi-tenant isolation. `user` (B2C students/teachers), `email-user` (email signups), `institute-user` (B2B institute admins/teachers). The validator **resolves the pool from the token's issuer** at runtime. Separate pools = independent password/MFA policies, separate signup flows, and blast-radius isolation between consumer and institutional identities.

Q6. How do service-to-service calls authenticate? (`@BypassJwtAuth`)

Answer. Trusted callers (CRM, internal jobs, webhooks) skip user-JWT via a **meta-annotation**.

`@BypassJwtAuth` marks other annotations (`@CrmserviceAPI` , `@InternalServiceAPI` , `@WebhookSecured`); the JWT filter's `shouldNotFilter()` sees it and steps aside, and a dedicated service filter validates a **shared service token** header instead.

```
@Target(ElementType.ANNOTATION_TYPE) @Retention(RUNTIME)
public @interface BypassJwtAuth { } // meta-annotation

@BypassJwtAuth @Target(METHOD) @Retention(RUNTIME)
public @interface CrmserviceAPI { } // put on CRM-callable endpoints
```

The shared base (`AbstractServiceAuthenticationFilter`) is a **template method**: check annotation applies → validate token exists → validate token value → optional extra check → authenticate as the service role. Each concrete filter supplies the header name, expected token, and `UserRole`.

PART B — AUTHORIZATION ("what can you do")

Q7. How is authorization enforced? (this is the differentiator)

Answer. Two layers — **defense in depth**:

1. **Declarative, endpoint-level** — `@PermissionContext(userPermission = {...})` on controller methods, enforced by a `HandlerInterceptor` (Order 7, right after JWT auth). It reads the required permissions from the annotation, loads the user's permissions from the DB (role→permission mapping, with role inheritance), and allows if the user has **any** required permission — else throws `SecurityPermissionException` (403).
2. **Imperative, in-code** — business rules call explicit checks like `instituteUser.validateAsTeacher()` when the rule is domain-specific.

The annotation (`common/annotation/PermissionContext.java`):

```
@Target(ElementType.METHOD) @Retention(RUNTIME)
public @interface PermissionContext { Permission[] userPermission(); }
```

The interceptor (`security/PermissionInterceptor.java` , `@Order(7)`):

```
public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) {
    if (shouldProceed(req, handler)) return true; // no annotation → skip
    JwtAuth user = (JwtAuth) req.getAttribute(LOGGED_USER_ATTRIBUTE);
    List<String> required = getRequiredPermissions(handler); // from
    @PermissionContext
    List<String> userPerms = fetchUserPermissions(user.getUserRole()); // walk role hierarchy
    in DB
    if (userPerms.stream().noneMatch(required::contains))
        throw new SecurityPermissionException(req.getRequestURI()); // 403
    return true;
}
```

Usage on an endpoint:

```
@PostMapping("/create-user")
@PermissionContext(userPermission = {Permission.AUTH_WRITE})
public ResponseEntity<String> createUser(@Valid @RequestBody CreateUserRequestDTO dto) { ... }
```

Q8. Explain the role & permission model.

Answer. Roles map to permissions in the DB (`role` , `role_permission` collections). A `Permission` enum enumerates fine-grained rights (`AUTH_READ/WRITE` , `EXERCISE_READ/WRITE` , `EXECUTE_SOLUTION_READ` , ...). A `UserRole` enum defines roles — and roles can **inherit** via a `derivedFrom` link, so the interceptor walks up the chain collecting permissions.

```
public enum UserRole {
    ADMIN, USER, CONTENT_CREATOR, SUPER_ADMIN, CONTENT_REVIEWER, PARENT,
    INSTITUTE_ADMIN(USER), INSTITUTE_TEACHER(USER), // inherit USER's permissions
    CRM_SERVICE_ACCOUNT, INTERNAL_SERVICE_ACCOUNT; // service accounts
    private final UserRole derivedFrom;
}
```

So `INSTITUTE_TEACHER` gets every `USER` permission **plus** its own — I don't duplicate grants. Permissions live in data (not code), so changing what a role can do is a DB change, not a redeploy.

Q9. Show an imperative authorization check.

Answer. For domain rules I check on the resolved `InstituteUser` (`domain/institute/model/InstituteUser.java`):

```
public boolean isTeacherOrAdmin() { return isTeacher() || isInstituteAdmin(); }
public void validateAsTeacher() {
    if (!isTeacherOrAdmin())
        throw new APIException(HttpStatus.UNAUTHORIZED, "This API is accessible only to teachers.");
}
```

Used in the solution-visibility service before any write:

```
teacher.validateAsTeacher(); // only teachers/admins may change visibility
```

That's the two-layer story: the interceptor guards the *endpoint by permission*; the service guards the *action by domain role*.

Q10. Refresh tokens & session control?

Answer. After login we issue a short-lived access JWT + a **refresh token** (stored, and in an `HttpOnly Secure` cookie for SSO). The frontend interceptor silently swaps an expired access token using the refresh token (see `01-frontend.md` Q9). Concurrent logins are capped by `MultiLoginDetectionFilter` — a second device can force-remove the first session, surfaced to the client via error code `60002`.

Summary — Authentication vs Authorization

	Authentication	Authorization
Question	who are you?	what can you do?
Where	<code>JwtAuthenticationFilter</code> (Order 6)	<code>PermissionInterceptor</code> (Order 7) + in-code checks
Input	JWT (Cognito RSA / SSO HMAC)	user role → permissions (DB)
Output	<code>SecurityContext</code> + request attributes	allow / 403
Data	Cognito (external) / <code>JwtService</code>	<code>role</code> , <code>role_permission</code> collections
Bypass	<code>@BypassJwtAuth</code> (service tokens)	endpoints without <code>@PermissionContext</code>
Annotation	—	<code>@PermissionContext(userPermission={...})</code>

One-line senior takeaway: "Stateless JWT authentication with dual token support (Cognito RSA + our SSO HMAC) across three Cognito pools, then defense-in-depth authorization — declarative `@PermissionContext` enforced by an interceptor over a DB-backed role/permission model with inheritance, plus explicit domain checks like `validateAsTeacher()`."

10. Annotations Reference — every annotation you use, with a real file

A quick-recall census of the annotations actually used in Robogebra. Interviewers love "what does this annotation do and where did you use it?" — this is your answer sheet. The **custom @interface annotations** at the end are your differentiator.

Backend — Spring Core / DI

Annotation	What it does	Used in
@RestController	@Controller + @ResponseBody — JSON REST endpoints	SolutionVisibilityController
@Service	business-logic bean	SolutionVisibilityService
@Repository	data-access bean (+ exception translation)	ExerciseSolutionRepository
@Component	generic Spring bean	JwtAuthenticationFilter
@Configuration	declares bean config	CacheConfig, MailConfig
@Bean	factory method → managed bean	OpenApiConfig, CacheConfig
@Autowired	inject dependency (we prefer constructor)	WebMvcConfig
@Value	inject a property value	GoogleAuthSuccessHandler
@ConfigurationProperties	bind a config block to a class	UsageLimitConfiguration
@Qualifier / @Primary	pick among multiple beans	FilterChainExceptionHandler / CognitoConfig
@Profile	bean only for a profile (local/ prod)	CognitoConfig
@Order	filter/interceptor/bean ordering	JwtAuthenticationFilter (6), PermissionInterceptor (7)
@ConditionalOnProperty	bean only if property set	CacheConfig, WebClientConfig
@EnableCaching / @EnableAsync	turn on caching / async	CacheConfig / RobogebraApplication

Backend — Web / REST

Annotation	What it does	Used in
<code>@RequestMapping</code>	base path for a controller	<code>SolutionStepFeedbackController</code>
<code>@GetMapping</code> / <code>@PostMapping</code> / <code>@PutMapping</code> / <code>@PatchMapping</code> / <code>@DeleteMapping</code>	map HTTP verbs	across controllers
<code>@PathVariable</code>	bind URL segment	<code>/exercise-items/{id}/...</code>
<code>@RequestParam</code>	bind query/form param	feedback/analytics controllers
<code>@RequestBody</code>	bind JSON body	<code>updateStatus(...)</code>
<code>@RequestAttribute</code>	bind value set by a filter (the authenticated user!)	<code>@RequestAttribute(INSTITUTE_USER_ATTRIBUTE)</code>
<code>@DateTimeFormat</code>	parse date params	<code>ParentAnalyticsController</code>

Backend — Validation

Annotation	What it does	Used in
<code>@Valid</code> / <code>@Validated</code>	trigger bean validation	<code>@Valid @RequestBody ...</code>
<code>@NotNull</code> / <code>@NotEmpty</code> / <code>@NotBlank</code>	presence rules	<code>SolutionVisibilityUpdateDTO</code> , <code>CreateInstituteRequestDTO</code>
<code>@Size</code> / <code>@Min</code> / <code>@Max</code>	length/range	<code>AdminSignUpRequestDTO</code> , <code>CacheProperties</code>
<code>@Email</code> / <code>@Pattern</code>	format rules	<code>ParentOtpVerifyRequestDTO</code> , <code>CreateInstituteRequestDTO</code>

Backend — Spring Data MongoDB

Annotation	What it does	Used in
<code>@Document</code>	class → Mongo collection	<code>ExerciseSolutionEntity</code>
<code>@Id</code>	primary key <code>_id</code>	every entity
<code>@Field</code>	custom BSON field name	entities
<code>@Indexed</code>	single-field index	<code>SolutionVisibilityEntity</code>
<code>@CompoundIndex(es)</code>	multi-field / unique index	<code>ExerciseEntity</code> , <code>SolutionVisibilityEntity</code>
<code>@DBRef</code>	reference another document	<code>ExerciseEntity.book/chapter</code>
<code>@CreatedDate</code> / <code>@LastModifiedDate</code>	audit timestamps	entities
<code>@Query</code> / <code>@Aggregation</code>	custom query / pipeline	<code>ExerciseRepository</code>
<code>@Transactional</code>	transactional boundary	write services

Backend — Lombok

Annotation	What it does	Used in
<code>@Getter</code> / <code>@Setter</code>	accessors	DTOs/entities everywhere
<code>@Builder</code>	fluent builder	models/DTOs
<code>@AllArgsConstructor</code>	all-args ctor (→ constructor injection)	services
<code>@NoArgsConstructor</code>	no-arg ctor (deserialization)	DTOs/entities
<code>@RequiredArgsConstructor</code>	ctor for <code>final</code> fields	<code>GoogleAuthSuccessHandler</code>
<code>@Data</code>	getters+setters>equals+hashCode+toString	<code>BitlyProperties</code>
<code>@Slf4j</code>	<code>log</code> field for SLF4J	filters/services

Backend — Exceptions, Async, Swagger

Annotation	What it does	Used in
<code>@ControllerAdvice</code> + <code>@ExceptionHandler</code>	global error handling	<code>ExceptionHandlerAdvice</code>
<code>@Async</code>	run on a thread pool	<code>NotificationService</code>
<code>@Tag</code> / <code>@Operation</code> / <code>@ApiResponse</code> / <code>@Schema</code>	OpenAPI/Swagger docs	controllers

Backend — CUSTOM annotations (your differentiator)

These are project-defined `@interface`s — mentioning them signals real senior ownership.

Authorization / auth-bypass:

```

@Target(ElementType.METHOD) @Retention(RUNTIME)
public @interface PermissionContext { Permission[] userPermission(); } // endpoint
permission gate

@Target(ElementType.ANNOTATION_TYPE) @Retention(RUNTIME)
public @interface BypassJwtAuth { } // meta-annotation

```

- `@PermissionContext(userPermission={...})` — declares required permissions; enforced by `PermissionInterceptor`.
- `@BypassJwtAuth` — meta-annotation; marks `@CrmServiceAPI`, `@InternalServiceAPI`, `@WebhookSecured` so those endpoints skip user-JWT and use a service token instead.
- `@BookAccessRequired` — gates endpoints behind subscription/book access (enforced by `BookAccessRestrictionFilter`).

Custom cross-cutting behavior: - `@FeatureUsageQuota` — endpoint counts against a plan quota (`FeatureUsageLimitFilter`). - `@PreferredLanguage` — apply the user's language (`PreferredLanguageFilter`). - `@NotificationEnabled` — method triggers a notification.

Custom validators (Bean Validation with `@Constraint`):

```

@Constraint(validatedBy = PasswordMatchesValidator.class)
@Target(ElementType.TYPE) @Retention(RUNTIME)
public @interface PasswordMatches { String message() default "Password does not match"; ... }

```

- `@PasswordMatches` — class-level cross-field check (password == confirm).
- `@ValidUserName` — field-level username format validator.

Talking point: "We built custom annotations to keep cross-cutting concerns declarative — `@PermissionContext` for authz, `@BypassJwtAuth` for service auth, `@FeatureUsageQuota` for plan limits, and `@PasswordMatches` / `@ValidUserName` custom validators. Each is backed by an interceptor, filter, or `ConstraintValidator`, so controllers stay clean and the intent is readable at the endpoint."

Frontend — Angular decorators

Decorator	What it does	Used in
<code>@Component</code>	component (selector/template/styles)	<code>app.component.ts</code>
<code>@NgModule</code>	module (declarations/imports/providers)	<code>app.module.ts</code>
<code>@Injectable</code>	injectable service (<code>providedIn: 'root'</code>)	<code>auth.service.ts</code> , interceptors
<code>@Directive</code>	custom directive	<code>graph-pan-zoom.directive.ts</code>
<code>@Input</code> / <code>@Output</code>	parent→child data / child→parent events	feature components (modals)
<code>@ViewChild</code> / <code>@ViewChildren</code>	reference child elements/components	<code>app.component.ts</code> , <code>solution-step.component.ts</code>
<code>@HostListener</code>	listen to host/window DOM events	<code>app.component.ts</code>
<code>@Inject</code> / <code>@Optional</code>	DI token / optional dependency	<code>app.component.ts</code> , <code>chapter-detail-modal</code>

30-second recall

- **DI:** `@Service/@Repository/@RestController` + constructor injection (`@AllArgsConstructor`).
- **REST:** `@GetMapping/@PostMapping/@PatchMapping` + `@PathVariable/@RequestBody/@RequestAttribute` .
- **Validation:** `@Valid` + `@NotNull/@NotEmpty/@Email/@Pattern` (+ custom `@PasswordMatches`).
- **Mongo:** `@Document/@Id/@Indexed/@CompoundIndex/@DBRef/@CreateDate` .
- **Errors:** `@ControllerAdvice` + `@ExceptionHandler` .
- **Custom:** `@PermissionContext` , `@BypassJwtAuth` , `@FeatureUsageQuota` .
- **Angular:** `@Component/@Injectable/@Input/@Output/@ViewChild/@HostListener` .

11. Testing — Interview Q&A tied to Robogeбра

Senior interviews always probe testing. Your project has ~32 backend test classes (JUnit 5 + Spring Boot Test + Testcontainers) and ~147 frontend .spec.ts files (Jasmine/Karma). That's a strong, honest story.

Q1. How do you test your Spring Boot backend?

Answer. Two levels: - **Unit tests** — a service in isolation with **Mockito** mocks for its collaborators; fast, no Spring context. Assert business logic and edge cases. - **Integration tests** — `@SpringBootTest` boots the app and hits real controllers through `MockMvc`, with a **real MongoDB in a Testcontainer** so I'm testing against actual Mongo behavior (indexes, queries), not an in-memory fake.

In Robogeбра. Integration tests live under `src/test/java/.../integration/` — e.g. `AuthControllerTest`, `ChapterControllerTest`, `ExerciseSolutionWorkoutControllerTest`, `UserControllerTest`. A `BaseBusinessTest` + `TestConfig` provide shared setup. `pom.xml` pulls `spring-boot-starter-test` (JUnit 5, Mockito, AssertJ) and `testcontainers:junit-jupiter`.

Unit test shape (Mockito):

```
@ExtendWith(MockitoExtension.class)
class SolutionVisibilityServiceTest {
    @Mock SolutionVisibilityPersistenceService persistence;
    @Mock StudentEnrollmentService enrollment;
    @InjectMocks SolutionVisibilityService service;

    @Test void studentWithNoClass_getsAccess() {
        when(enrollment.classIds(any())).thenReturn(List.of());
        assertTrue(service.isSolutionEnabledForStudent(student, "item-1")); // default open
        verify(persistence, never()).findByClassIdsAndTargetIds(any(), any());
    }
}
```

Integration test shape (Testcontainers + MockMvc):

```
@SpringBootTest @AutoConfigureMockMvc @Testcontainers
class ChapterControllerTest {
    @Container static MongoDBContainer mongo = new MongoDBContainer("mongo:6");
    @DynamicPropertySource static void props(DynamicPropertyRegistry r) {
        r.add("spring.data.mongodb.uri", mongo::getReplicaSetUrl);
    }
    @Autowired MockMvc mvc;
    @Test void getChapters_returns200() throws Exception {
        mvc.perform(get("/api/chapters/...").header("Authorization", "Bearer " + token))
            .andExpect(status().isOk());
    }
}
```

Q2. Unit vs integration — when each?

Answer. **Unit** for logic-heavy code with branches (the visibility precedence resolver, validators, mappers) — fast, deterministic, run on every save. **Integration** for wiring I can't fake safely — actual Mongo queries/

indexes, the security filter chain, serialization, and end-to-end controller behavior. I aim for many fast unit tests + a focused set of integration tests on critical paths (auth, solutions, payments).

Why Testcontainers over an embedded/mock Mongo? It runs the *real* MongoDB in Docker, so unique indexes, `@DBRef`, and aggregation pipelines behave exactly as in prod. Embedded fakes drift from real behavior and give false confidence.

Q3. How do you mock, and what do you assert?

Answer. Mockito `@Mock` + `when(...).thenReturn(...)` to stub collaborators, `verify(...)` to assert interactions (e.g. "we did NOT hit the DB when the student has no class"). I assert **behavior + outputs + edge cases** (null/empty, unauthorized, boundary), not just the happy path. For exceptions I use `assertThrows(APIException.class, ...)` and check the status/message.

Q4. How do you test the Angular frontend?

Answer. Jasmine specs run by Karma, using Angular's `TestBed`. Components are tested with mocked services (spy objects); services with `HttpTestingController` to assert the right requests and simulate error responses; guards/resolvers with mocked router + service. RxJS is tested with marble-style or `fakeAsync / tick`.

In Robogebra. ~147 `.spec.ts` files (co-located `*.component.spec.ts`). Example patterns I'd describe: - A guard test: mock `AuthService.isLoggedInObservable()` → assert `canActivate` redirects to `/login` and returns false when logged out. - A service test: `HttpTestingController` — call `authService.createUser(...)`, expect a `POST`, flush a `409` and assert the error path fires the notification.

```
it('redirects to login when not authenticated', (done) => {
  authService.isLoggedInObservable.and.returnValue(of(false));
  guard.canActivate(route, state).subscribe(allowed => {
    expect(allowed).toBeFalse();
    expect(router.navigate).toHaveBeenCalledWith(['/login']);
    done();
  });
});
```

Q5. What's your overall test strategy / philosophy?

Answer. Test pyramid — lots of fast unit tests, fewer integration tests, minimal E2E. Test **behavior, not implementation**, so refactors don't break tests. Cover the **risk**: auth, authorization, money, and the tricky logic (visibility precedence). Tests run in **CI on every PR** (the repo has a `.github` pipeline + Dockerfile), so nothing merges red. I don't chase 100% coverage — I chase coverage of the code that would hurt if it broke.

Honest note. Coverage isn't uniform across 60+ domains; critical paths (auth, exercise solutions, chapters, users) have integration tests, and I'd expand coverage on newer domains. Saying that is more credible than claiming everything is tested.

Rapid-fire recap

Topic	Robogebra reality
Backend unit	JUnit 5 + Mockito (<code>@Mock</code> / <code>@InjectMocks</code>)
Backend integration	<code>@SpringBootTest</code> + <code>MockMvc</code> + Testcontainers Mongo
Test count	~32 backend classes, ~147 frontend specs
Frontend	Jasmine + Karma + <code>TestBed</code> + <code>HttpTestingController</code>
Philosophy	pyramid, behavior-focused, risk-first, CI on every PR

12. System Design — walkthroughs tied to Robogebra

Senior rounds ask you to design something on a whiteboard. Best strategy: **design a feature you actually built**. These are ready-to-draw walkthroughs.

Design 1 — Solution Visibility (the feature you built end-to-end)

Prompt framing: "Teachers want to control when their class can see a solution — at chapter, exercise, or single-question level. Design it."

Requirements. - Teacher enables/disables a solution for chosen classes, at 3 granularities (chapter / exercise / question). - Student's app checks access before showing a solution. - Most-specific rule wins; default = visible; students outside an institute unaffected.

Data model (MongoDB). One collection `solution_visibility`, one row per (class, target):

```
{ klassId, targetId, status: ENABLED|DISABLED, createdAt, updatedAt }  
unique compound index (klass_id, target_id) // one rule per class per target → upsert, no  
dups
```

Write path (teacher). `PATCH /api/solution-visibility/status {level, targetId, klassIds[], status}` → validate teacher role → validate target exists → **upsert** one row per class.

Read path (student). `GET /exercise-items/{id}/solution-access` → 1. Resolve student's `klassIds` (enrollment). 2. Build precedence `[itemId, exerciseId, chapterId]`. 3. **One** query for all classes × all three targets; `groupBy(targetId)`. 4. Walk precedence most-specific-first; first level with a rule decides (`anyMatch ENABLED`); else open.

Why these choices (say this). Unique index = data-integrity + upsert semantics. Single query + in-memory grouping = one round trip, O(1) lookups (vs N queries). Precedence list = the "specific overrides general" product rule expressed as data. Default-open + no-class-skip = safe for non-institute users.

Scale/extend. Cache per-class rules (rarely change); add an exercise-level access endpoint by reusing the precedence resolver; audit via timestamps.

```
Teacher UI —PATCH status→ Portal —upsert(klass,target)→ solution_visibility  
Student UI —GET access→ Portal —1 query [item,ex,chapter]→ precedence walk → allowed?
```

Design 2 — File upload to S3 (classic full-stack design)

Prompt: "Users upload images/assets. Design upload + serving."

Answer — pre-signed URLs, file never touches the app tier. 1. Client asks portal: "I want to upload X" → portal returns a **pre-signed PUT URL** (short expiry) + the object key. 2. Client **PUT**s the bytes **directly to S3**. 3. Client tells portal the key → portal stores **only key + metadata** in MongoDB (not the binary). 4. To display, portal returns a pre-signed **GET** URL (5-min expiry).

Why: keeps large binaries out of the app server and the DB, scales with S3, and stays private (no public buckets). Add CloudFront for delivery, validate content-type/size, virus-scan async via Lambda if needed.

This is exactly Robogeбра's model ([robogeбра-dev-assets](#) / [robogeбра-core-content](#) , 5-min signed URLs).

```
Client -req URL→ Portal -presign→ (PUT direct)→ S3
Client -save key→ Portal → Mongo{key,meta}; read: Portal -presigned GET→ Client
```

Design 3 — Notifications (multi-channel, async)

Prompt: "Send OTP / payment / welcome messages over email, SMS, WhatsApp."

Answer — Factory + async. A `NotificationServiceFactory` picks the channel (WhatsApp if the username is a phone number, else email via SES), and methods are `@Async` so the request thread never blocks on a third-party call. Real Robogeбра code:

```
@Async public void sendOtp(String userName, String otp)
{ getNotificationService(userName).sendOTP(userName, otp); }
private NotificationService getNotificationService(String u) {
    return u.matches(PHONE_PATTERN) ? whatsappNotificationService : emailNotificationService;
}
```

Scale: move to a queue (SNS/SQS) for retries + backpressure; make sends idempotent; dead-letter failures.

Design 4 — "How would you scale Robogeбра for 10x users?"

Answer — walk the tiers. - **App tier:** services are **stateless** (JWT, no session) → horizontal autoscale behind a load balancer. - **DB tier:** Mongo — right indexes (compound, high-cardinality), read replicas, shard hot collections; watch `@DBRef` N+1 (prefer `$lookup` /embed for hot reads). - **Cache:** Caffeine in-process for rarely-changing content (books/chapters) with TTLs; Redis if we need a shared cache. - **Media:** S3 + CloudFront CDN. - **Async:** offload spiky/ML work to Lambda + queues; notifications async. - **Resilience:** timeouts + retries + circuit breakers on portal→CRM / portal→AI; graceful degradation (cached/basic answer if AI down). - **Observability:** metrics, logs, PostHog; capacity alarms.

Design framework (say this out loud in any design round)

1. **Clarify** requirements + scale (reads vs writes, consistency needs).
2. **Data model** first (it drives everything).
3. **APIs** (endpoints, verbs, status codes).
4. **Core algorithm** (e.g., the precedence resolution).
5. **Scale & failure** (cache, index, async, circuit breakers, degradation).
6. **Trade-offs** — always name what you gave up (e.g., Mongo flexibility vs multi-doc ACID).

13. Performance & Caching — Q&A tied to Robogeбра

Depth questions on "how do you make it fast" — answered with real project levers: Caffeine cache, Mongo indexing, pagination, `@DBRef` N+1, and frontend perf.

Q1. How do you cache, and what do you cache?

Answer. In-process **Caffeine** cache (Spring cache abstraction) for data that's **read-often, changes-rarely** — books, chapters, exercises. Per-group **TTL + max size**, and the whole cache is **toggleable by property** (so we can disable it per environment). Content that rarely changes but is read on every page load is the ideal cache candidate; user-specific/volatile data is not.

In Robogeбра (`CacheConfig`) — conditional Caffeine manager with per-group TTL:

```
@Configuration @EnableCaching
public class CacheConfig {
    @Bean @ConditionalOnProperty(prefix="cache", name="enabled", havingValue="true")
    public CacheManager cacheManager() {
        List<CaffeineCache> caches = cacheProperties.getManager().stream()
            .flatMap(g -> g.getCacheNames().stream().map(name -> new CaffeineCache(name,
                Caffeine.newBuilder().expireAfterWrite(g.getTtlMinutes(), TimeUnit.MINUTES)
                    .maximumSize(g.getMaxSize()).recordStats().build())))
            .toList();
        ...
    }
    @Bean @ConditionalOnProperty(prefix="cache", name="enabled", havingValue="false")
    public CacheManager noOpCacheManager() { return new NoOpCacheManager(); }
}
```

TTLs in config: ~1440 min (24h) for books/chapters/exercises, ~60 min for subscriptions, max ~10k entries.

Follow-up: cache invalidation? The two hard problems in CS 😊. We use **TTL (expire-after-write)** as the baseline, and `@CacheEvict` on writes for correctness where staleness matters. In-process Caffeine is per-instance; for a shared cache across instances I'd move to **Redis**.

Q2. How do you keep MongoDB queries fast?

Answer. **Indexes matched to query patterns**, one round-trip designs, and pagination. `@Indexed` on filtered fields, `@CompoundIndex` for multi-field queries and uniqueness (order matches query + sort). I avoid N+1 by fetching in bulk.

In Robogeбра. - Unique compound `(klass_id, target_id)` on visibility — fast lookups + integrity. - `(chapter.$id, order)` on exercises so "chapter's exercises, sorted" is index-backed. - The visibility check does **one** `findByKlassIdInAndTargetIdIn(...)` for all classes × targets instead of N queries (see `05-database.md` Q5).

Follow-up: `@DBRef` N+1 risk? `@DBRef` resolves with extra queries, so a list of exercises each dereferencing book/chapter can explode into N+1. On hot read paths I'd store just the id or use an `$lookup` aggregation to join server-side.

Q3. How do you handle large result sets?

Answer. **Pagination** with Spring Data `Pageable` / `PageRequest` — never return unbounded lists. The repository takes a `Pageable`, returns a `Page<T>`, and I map entities→models while preserving page metadata.

In Robogebra (`SolutionStepFeedbackService`) — paged query + `Page.map` :

```
return solutionStepFeedbackRepository
    .findByReviewedUser_Id(userId, pageable)    // Pageable in
    .map(SolutionStepFeedbackEntity::toModel);  // Page<Model> out, metadata kept
```

`PageableConstants` centralizes default page sizes. Controllers accept page/size params and pass a `PageRequest` down.

Q4. Async & non-blocking — where does it help perf?

Answer. Move slow, non-critical I/O **off the request thread** with `@Async` so response latency isn't hostage to a third party. Notifications (OTP/email/WhatsApp), last-active-timestamp updates, and analytics writes are async. For outbound HTTP we use non-blocking `WebClient` with timeouts. Heavy/spiky compute (AI solution generation, interactive graphs) is offloaded to **Lambda** so it scales independently.

Q5. Frontend performance levers?

Answer. - `trackBy` on `*ngFor` → Angular reuses DOM nodes instead of destroy/recreate (used in analytics/progress lists). - **async pipe** → no manual subscriptions, fewer leaks, CD tied to emissions. - **Lazy-loaded** feature modules → smaller initial bundle. - `debounceTime` on rapid input → fewer expensive calls (worksheet executor). - **OnPush** + **immutability** (improvement I'd push) → skip dirty-checking untouched subtrees. - **Resolvers** → data ready on navigation, no layout thrash. - Silent **token refresh** in the interceptor → no failed-request round-trips visible to the user.

Q6. How would you find and fix a performance problem?

Answer. Measure first. Backend: enable Mongo profiler / `explain()` to find collection scans → add/adjust indexes; check for N+1 from `@DBRef`; add caching for hot reads; paginate. Frontend: Chrome DevTools performance + Angular DevTools to find excessive change detection or large bundles → OnPush, `trackBy`, lazy-load, code-split. I optimize the **proven** bottleneck, not a guess, and confirm with before/after numbers.

Rapid-fire recap

Lever	Robogebra implementation	File
Caching	Caffeine, per-group TTL, toggleable, NoOp fallback	<code>CacheConfig.java</code>
Indexing	unique compound <code>(klass_id,target_id)</code> , <code>(chapter,order)</code>	entities
One round-trip	<code>findByKlassIdInAndTargetIdIn</code> bulk fetch	<code>SolutionVisibilityService.java</code>
Pagination	<code>Pageable</code> → <code>Page.map(toModel)</code>	<code>SolutionStepFeedbackService.java</code>
Async	<code>@Async</code> notifications, <code>WebClient</code> , Lambda offload	<code>NotificationServiceFactory.java</code>
Frontend	<code>trackBy</code> , <code>async</code> , lazy-load, <code>debounceTime</code>	lists, interceptor

14. Behavioral, Project Pitch & Delivery — tied to Robogeбра

Every interview opens and closes here. Prepare these until they're smooth. Fill [...] with your specifics (years, team size, etc.).

1. 60-second self-introduction (template)

"I'm Priyanka, a full-stack engineer with [X] years building web applications, primarily **Angular on the front end and Java/Spring Boot with MongoDB on the back end**. Currently I work on **Robogeбра**, an ed-tech learning platform for students and coaching institutes. I work across the stack — the **Angular student/teacher web app** and the **Spring Boot backend** — on features like solution management, a teacher-controlled **solution-visibility** system, authentication with AWS Cognito, and subscription flows. I care about clean, layered code, and I've owned features end to end: from the MongoDB data model and REST API through to the Angular UI. I'm looking for a role where I can keep growing as a senior full-stack engineer."

Keep it: who → stack → current project → what you own → what you want.

2. 60-second project pitch (Robogeбра)

"Robogeбра is a learning platform with a two-sided market — individual students and coaching institutes. Architecturally it's a **microservices hybrid**: a Spring Boot **portal** is the monolithic learning core (books → chapters → exercises → solutions), a separate **CRM** service handles institutes/coupons/subscriptions, and there are multiple frontends — an Angular web app, an Ionic mobile app, and a React CRM. Data is in **MongoDB**, auth is **AWS Cognito with JWT** across three user pools, and we use S3 for assets. I've worked mostly on the portal and the Angular app — recently I built the **solution-visibility feature**, where teachers control when their classes can see solutions at chapter, exercise, or question level."

3. STAR stories (use REAL features you built)

STAR = Situation, Task, Action, Result. Two solid stories beat ten vague ones.

Story A — "A challenging feature you built" → Solution Visibility

- **S:** Teachers wanted to stop students seeing solutions before they'd attempted problems, but control needed to work at three levels — whole chapter, an exercise, or a single question.
- **T:** Design and build it end-to-end — data model, backend rules, and the UI — without slowing down the student's solution page.
- **A:** Modeled it as one `solution_visibility` document per (class, target) with a **unique compound index** so enable/disable is a clean upsert. For the student check I resolved a **precedence** — question → exercise → chapter, most-specific wins, default open — and did it in

one MongoDB query by fetching all candidate rows and grouping in memory, so no extra latency. On the frontend I added a lock indicator with a hover tooltip and reused the existing per-item control.

- **R:** Teachers can now gate solutions per class at any level; the access check is a single round-trip; students outside an institute are never affected. It shipped cleanly and the precedence design made later extension (exercise-level gating) a small change.

Story B — "A tricky bug / production issue" → (adapt to a real one)

- Pick a real incident: e.g. a **token-expiry** causing failed requests. **A:** added silent token refresh in the HTTP interceptor (`switchMap` retry of the original request with the new token) + session-expired handling. **R:** users stopped getting random 401s; sessions refresh transparently.

Story C — "Disagreement / ownership"

- A design call where you pushed for the layered Entity↔Model↔DTO separation (or the extra persistence-service layer) for maintainability, explained the trade-off, and the team adopted it.

Prep 3–4 short stories; you can flex them to "challenge / conflict / failure / proud-of / learned-something."

4. Common behavioral questions — crisp answers

- **Biggest strength:** owning features end-to-end across the stack + writing clean, layered code others can maintain.
- **Weakness (honest + improving):** "I've been deep in Angular 16 with zone.js; I'm actively learning signals/zoneless and standalone components to stay current." (True for your project — see [01-frontend.md](#).)
- **How do you handle tight deadlines:** scope to the critical path, communicate trade-offs early, ship a solid MVP, iterate.
- **How do you keep learning:** building features that stretch me (auth, caching), reading the framework docs, and code review.
- **Why leaving / why this role:** growth into senior full-stack ownership and [company-specific reason].

5. Senior "ownership" talking points (sprinkle these)

- "I think in **trade-offs** — e.g. MongoDB gives us schema flexibility for varied content but not multi-doc ACID, so money flows stay in one service."
- "I design for **failure** — timeouts/retries on external calls, graceful degradation if the AI service is down."
- "I keep **cross-cutting concerns declarative** — custom annotations (`@PermissionContext` , `@FeatureUsageQuota`) backed by filters/interceptors."
- "I care about the **whole request lifecycle** — from the Angular guard/interceptor to the Spring filter chain to the Mongo index."

6. Git & delivery (be ready for the "how do you ship" question)

- **Branching:** feature branches off `main` / `develop`, small PRs, code review before merge.
 - **CI/CD:** the repo has a `.github` pipeline and a `Dockerfile` — build + tests run on every PR; nothing merges red; containerized deploys.
 - **Environments:** `application-dev.yml` / env vars (`${VAR:default}`) keep the same artifact deployable across dev/prod with different secrets.
 - **Quality gate:** integration tests (Testcontainers) + frontend specs run in CI (see `11-testing.md`).
-

7. Smart questions to ask the interviewer

- "How is the team structured, and how do features move from idea to production?"
 - "What does the testing / CI culture look like here?"
 - "What are the biggest technical challenges the team is facing right now?"
 - "What does growth look like for a senior engineer on this team?"
 - "How do you balance shipping speed with code quality / tech debt?"
-

8. Logistics (have answers ready)

- **Notice period / availability:** [your notice].
 - **Salary expectation:** give a researched range; "based on my experience and the market for senior full-stack roles, I'm looking at [range], but I'm open to discussing the whole package."
 - **Location / remote:** [your preference].
-

The night-before checklist

- [] 60-sec intro + project pitch, smooth without notes
- [] 3 STAR stories ready (solution-visibility is your strongest)
- [] Can whiteboard the architecture (`06-microservices.md`) and the S3 upload (`12-system-design.md`)
- [] Can explain memory leaks / `takeUntil` (`01`), auth chain + `@PermissionContext` (`09`), error handling (`08`)
- [] Honest on zoneless/signals, test coverage, Mongo trade-offs — reality + direction
- [] 2–3 questions ready for them